Intelligent Multi-Pattern Search and Document Analytics Engine

An intelligent platform for efficient document search, comparison, optimization and analysis

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

Bachelor of Technology

in the Department of Computer Science & Engineering

DATA STRUCTURES AND ALGORITHMS - 3 - 25CS2103E

Submitted by

2510030088 M Ruchee
2510030016 Oddiraj Sridhruthi
2510030093 K Poojasri Reddy

Under the guidance of
Dr. Anitha Patil



Department of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation (KLH), Aziz Nagar

October 2026

Abstract

The Intelligent Multi-Pattern Search and Document Analytics Engine is a browser-oriented project that brings together document ingestion, exact pattern search, document comparison, dynamic-programming optimization, network planning, and performance insights in one interface. The supplied project materials describe a pipeline that moves from upload and reading through tokenization and normalization, method selection, algorithm execution, result generation, and visualization. The architecture groups capabilities into document management, a search engine, a comparison engine, an optimization engine, a network planner, and analytics. The presentation lists string-search methods including Naive matching, KMP, Rabin–Karp, the Z-algorithm, suffix arrays and LCP; comparison methods including edit distance, LCS, Needleman–Wunsch and Smith–Waterman; dynamic-programming topics such as interval DP, knapsack, TSP and matrix-chain multiplication; and flow and matching techniques. The technology slide lists React with TypeScript, Vite, Tailwind CSS, Lucide React, React Router and Browser File APIs. Screenshots show the dashboard, a Smart Search interface with a Z-algorithm selection, and a Document Compare & Alignment interface. The project presentation reports a successful production build and verification of search, DP/optimization, flow/matching, navigation and responsive behavior. This report documents the design context, algorithmic background, reported verification, and a proposed reproducible evaluation plan. It does not claim independent code review, quantitative benchmarking, complete test coverage, or production-scale validation because those artifacts were not supplied. Any numbers visible in application screenshots are treated as example-screen values, not general performance results.

Keywords: document search, string matching, edit distance, dynamic programming, network flow, document analytics, algorithm visualization

> **Evidence note:** This report is based on the supplied presentation and its screenshots. Project-reported test statements are distinguished from recommendations and theory; displayed UI numbers are not presented as reproducible benchmarks.

List of Figures

Figure 2.1. Methodology pipeline and listed technology stack from the project presentation.
Figure 2.2. System architecture and functional engines from the project presentation.
Figure 2.3. Illustrative edit-distance dynamic-programming table for CAT and CUT.
Figure 2.4. Illustrative source–sink network for explaining capacity and flow.
Figure 3.1. Proposed repeatable verification cycle (recommended process, not completed test evidence).
Figure 4.1. Dashboard screenshot supplied with the project presentation.
Figure 4.2. Smart Search screen supplied with the project presentation.
Figure 4.3. Document Compare & Alignment screen supplied with the project presentation.

List of Tables

Table 1.1. Project objectives and corresponding evidence in supplied materials.

Table 2.1. Search-algorithm trade-offs described in the project context.

Table 2.2. Asymptotic complexity summary for the listed algorithms.

Table 2.3. Document-comparison methods and their interpretation.

Table 2.4. Dynamic-programming and network-planning problem families.

Table 3.1. Project-reported verification statements and missing evidence.

Table 3.2. Proposed functional test cases and acceptance checks.

Table 3.3. Proposed benchmark inputs and measurement protocol.

Table 4.1. Results evidence by project module.

Table 4.2. Claims supported by the materials versus claims requiring more evidence.

Table 5.1. Future-work roadmap transcribed from the source presentation.

Table C.1. Glossary of core terms.

Table of Contents

1. INTRODUCTION
    1.1 Problem statement
    1.2 Objectives and scope
    1.3 Constraints, stakeholders and requirements
    1.4 Success criteria and report organization
2. METHODOLOGY AND SYSTEM DESIGN
    2.1 Related work and technical context
    2.2 Pipeline and system architecture
    2.3 Requirements and text preparation
    2.4 String-search methods
    2.5 Comparison and alignment methods
    2.6 Optimization and network planning
    2.7 Selection, analytics and engineering constraints
3. EXPERIMENTAL DESIGN AND VERIFICATION
    3.1 Evidence boundaries
    3.2 Test strategy and cases
    3.3 Proposed benchmark protocol
    3.4 Missing artifacts and validity threats
4. RESULTS AND DISCUSSION
    4.1 Reported verification
    4.2 Interface evidence
    4.3 What the materials support and do not support
5. CONCLUSION AND FUTURE WORK

References

Appendix A. Algorithm outlines

Appendix B. User workflow guide

Appendix C. Test checklist and glossary

1. INTRODUCTION

This report documents the Intelligent Multi-Pattern Search and Document Analytics Engine using the supplied project presentation as its primary source. It summarizes the project purpose, system scope, algorithmic background, reported testing statements, and the limits of the available evidence. Intelligent Multi-Pattern Search and Document Analytics Engine — An intelligent platform for efficient document search, comparison, optimization and analysis. The project team comprises three students from the Department of Computer Science & Engineering at KLH University / Koneru Lakshmaiah Education Foundation (KLH), Aziz Nagar: M Ruchee (2510030088), Oddiraj Sridhruthi (2510030016), and K Poojasri Reddy (2510030093). The following sections draw exclusively on the supplied project facts and on standard algorithmic literature cited in the project materials. Where the project materials list algorithms, implementation components, or screenshots, those are treated as project-supplied facts; where the presentation provides verification statements, those are quoted with explicit limits on the evidence.

The engine integrates multiple algorithmic families and user-facing functions to support document ingestion, search, comparison, optimization and analytics. It aims to combine classical string-processing and graph/flow algorithms with a browser-based user interface to provide an interactive pipeline for document analysis and iterative improvement. The pipeline depicted in the project materials proceeds as: upload → read → tokenize → normalize → select method → execute algorithm → generate result → visualize, with iterative improvement. The platform's architecture is modular and comprises document management, search engine, comparison engine, optimization engine, network planner, and analytics components. The project materials list a front-end technology stack and a set of algorithmic techniques that inform the system's capabilities. The implementation materials describe six user-facing functions: Documents, Smart Search, Document Compare, Processing Optimizer, Network Planner, and Insights & Performance.

This report is organized as follows: Section 1 defines the problem, objectives, scope, stakeholders and requirements; Section 2 presents related work, system design and algorithmic foundations; Section 3 records reported verification and proposes a reproducible evaluation plan; Section 4 interprets the supplied results and screenshots; and Section 5 concludes with a roadmap. The appendices include reference pseudocode, a cautious workflow guide, test checklist and glossary.

1.1 Problem statement

Large collections of textual documents used in academic, administrative, and operational contexts require efficient, flexible methods for locating patterns, comparing document variants, optimizing document-processing workflows, and planning resources that may be represented as networks. The supplied project materials identify a need for an integrated platform that supports:

- ingesting and previewing documents in a browser context,
- tokenizing and normalizing textual content,
- performing single- and multi-pattern search with selectable algorithms,
- comparing documents via edit-distance, longest common subsequence, and sequence alignment,
- performing optimization analyses using dynamic programming formulations,
- planning network and resource flows with max-flow/min-cut and matching algorithms,
- delivering analytics and performance measurements to guide iterative improvement.

This combination of capabilities—pattern search, sequence comparison, optimization, network planning, and analytics—poses challenges in algorithm selection, interface design for algorithm choice and parameterization, and producing interpretable visualizations of algorithmic outputs that support iterative workflows.

1.2 Project objectives

Table 1.1. Project objectives and evidence in supplied materials.

Objective	Project intention	Evidence in supplied materials
Document handling	Upload, read, preview, tokenize and normalize documents	Documents module and pipeline slide
Smart search	Single-/multi-pattern exact search with multiple algorithm choices	Smart Search screen and algorithms list
Document comparison	Edit distance, common subsequence, global/local alignment	Document Compare screen
Processing optimization	Dynamic-programming workflows and optimization topics	Processing Optimizer module
Network planning	Flow, bottleneck, matching and assignment problems	Network Planner module
Insights	Present actual outputs and performance measurements	Insights & Performance module; no raw benchmark logs supplied

The project materials state the following objective explicitly: integrate document ingestion/preview/tokenization/normalization; single and multi-pattern search; document comparison with edit distance, common subsequence, sequence alignment; workflow optimization using dynamic programming; network/resource

planning; analytics and performance measurements. From that statement, the project objectives can be articulated as concrete goals:

- Provide a browser-based pipeline allowing users to upload documents, preview content, and perform tokenization and normalization in preparation for algorithmic processing.
- Implement selectable string-search algorithms (the project materials list Naive, Knuth–Morris–Pratt, Rabin–Karp, Z-algorithm, suffix array and LCP) accessible through a Smart Search interface.
- Implement document comparison methods including edit distance (Wagner–Fischer), longest common subsequence, global alignment (Needleman–Wunsch), and local alignment (Smith–Waterman), accessible through a Document Compare interface that supports alignment visualization.
- Provide optimization modules exposing dynamic programming techniques such as interval DP, knapsack DP, traveling salesman problem (TSP) formulations, and matrix chain multiplication to support workflow optimization problems derived from document processing tasks.
- Provide network planning functionality implementing flow and matching algorithms (Ford–Fulkerson, Edmonds–Karp, max-flow/min-cut, bipartite matching, assignment) to support resource allocation and routing analyses.
- Produce analytics and performance instrumentation that record algorithmic behavior and provide actionable insights to users via a dashboard; support iterative improvement of inputs and algorithm choices.
- Deliver an integrated front-end experience that is responsive and usable in modern browsers, leveraging the project materials' front-end technology stack.

1.3 Scope

The project materials define a scope that is focused on a browser-based, front-end-driven platform with a modular algorithmic core. The scope includes:

- Client-side document ingestion using browser File APIs: uploading, reading, previewing.
- Tokenization and normalization operations in the front-end pipeline to prepare text for search and comparison algorithms.
- Implementation and integration of the listed search, comparison, optimization, and network algorithms into the front-end UI such that a user can select methods, set parameters, execute algorithms, and view results and visualizations.
- A navigation structure and responsive UI with dedicated screens or views for Documents, Smart Search, Document Compare, Processing Optimizer, Network Planner, and Insights & Performance as described in the implementation slide and screenshots.

The following are explicitly outside the scope given the supplied facts:

- Any backend server-side components, persistent storage, databases, or deployment infrastructure details are not supplied in the project materials and thus are not included in the scope description here.
- Performance claims that are not present in the supplied materials (e.g., benchmark figures, throughput, or comparative timings) are not asserted.
- Any environment-specific software versions, continuous integration configuration, or test logs absent from the project materials are not described.

1.4 Constraints and assumptions

The project materials impose or imply the following constraints and assumptions that affect design and implementation choices:

- Software execution is expected in a modern browser environment. The technology-stack slide lists React + TypeScript, Vite, Tailwind CSS, Lucide React, React Router, Browser File APIs; it also mentions Node.js and npm as software requirements. No versions were supplied, and no server-side components or persistent storage were specified in the materials.
- The listed front-end technologies suggest a browser-oriented design; any client-side implementation would be constrained to what is feasible in client-side JavaScript/TypeScript running in the browser (memory and CPU limits of client machines are practical constraints not quantified in the materials).
- The project materials list algorithmic techniques and claim that certain algorithm families were "tested" or "confirmed." The presentation reported that the production build was verified PASS, string-matching algorithms tested, DP and optimization modules confirmed, flow and graph algorithms confirmed, routes/navigation tested, and responsive UI verified. However, the presentation did not supply environment versions, test logs, test counts, coverage percentages, benchmark protocols, or independent verification artifacts. These verification statements should therefore be treated as project-reported outcomes without external validation.
- Screenshots were supplied for the dashboard, Smart Search (with a selected Z-algorithm and an example query), and Document Compare & Alignment (with options for edit-distance/LCS/global/local alignment). Those screenshots are

evidence of interface states; any displayed values are screen-specific illustrative values and must not be generalized as performance or accuracy claims.

- The project materials list algorithms by name but do not claim that every listed algorithm is proven in code within the delivered artifacts. When describing the implementation status, the report will refer to "the project materials list" rather than asserting full implementation where not documented.

1.5 Stakeholders and user needs

Primary stakeholders identified implicitly by the project scope are:

- End users seeking efficient document search, comparison, and analytics functionality in a browser-based environment. Their needs include rapid pattern matching across documents, clear presentation of matches and alignments, and the ability to iterate on queries and document variants.
- Users responsible for planning document-processing workflows or resource allocation who need optimization and network-planning tools to evaluate trade-offs, schedule tasks, or model resource flows.
- The project team and course instructors, who require clear deliverables demonstrating algorithmic understanding, an integrated user interface, and evidence that the implemented components function as intended.

User needs that the platform aims to satisfy, as derived from the project materials, include:

- A simple upload and preview mechanism for working with local files in the browser.
- Tokenization and normalization utilities to convert raw document text into forms suitable for pattern matching and comparison.
- Selectable search algorithms so that users can choose the algorithmic approach (e.g., KMP vs. Rabin–Karp) appropriate for a query or educational purpose.
- Visual, interpretable outputs for comparisons such as edit-distance alignments and LCS highlighting.
- Optimization and network analysis tools that can be configured to model real workflow or resource constraints and produce recommendations or visual plans.
- Dashboard-level analytics presenting insights and performance metrics that guide iterative refinement of queries, documents, and algorithm parameters.

1.6 Requirements

From the project objectives, scope, and stakeholders' needs, the following functional and non-functional requirements can be stated. These are derived from the project materials and phrased in an evidence-bound manner.

Functional requirements (high level):

- FR1: The system shall allow users to upload documents via the browser and preview their contents.
- FR2: The system shall provide tokenization and normalization operations on uploaded documents.
- FR3: The system shall expose a Smart Search interface where users can select among the project-listed string-search algorithms and execute single- and multi-pattern searches on tokenized/normalized text.
- FR4: The system shall provide a Document Compare interface offering edit distance (Wagner–Fischer), longest common subsequence, global alignment (Needleman–Wunsch), and local alignment (Smith–Waterman) options, with alignment visualization.
- FR5: The system shall provide a Processing Optimizer that implements dynamic programming techniques for workflow optimization tasks (interval DP, knapsack DP, TSP, matrix chain).
- FR6: The system shall provide a Network Planner that implements flow and matching algorithms (Ford–Fulkerson, Edmonds–Karp, max-flow/min-cut, bipartite matching, assignment) for resource planning tasks.
- FR7: The system shall collect analytics and basic performance measurements and present them via an Insights & Performance screen or dashboard.

Non-functional requirements and constraints:

- NFR1: The user interface shall be implemented using the project-supplied front-end technologies (React + TypeScript, Vite, Tailwind CSS, Lucide React, React Router) as listed in the project materials.
- NFR2: The system shall operate in a modern browser environment using Browser File APIs for file ingestion. Node.js and npm are required for development tooling as stated in the project materials.
- NFR3: The UI shall be responsive and navigable (the project materials state the responsive UI and routes/navigation were tested).
- NFR4: The system shall make algorithmic choices explicit and transparent to the user to support educational and analytical uses; users shall be able to select the algorithm and view selected parameters before execution.
- NFR5: The system's analytics and performance claims shall be recorded and presented factually; any verification statements provided by the project materials shall be identified as project-reported and limited by the absence of formal benchmarks or independent verification.

1.7 Success criteria and verification limits

The project materials include statements that a production build was verified PASS, various algorithm families were tested or confirmed, and the UI responsiveness and navigation were tested. These statements provide a basis for internal success criteria but need to be qualified in a technical report:

- Internal verification criteria (derived from the project materials):
- The production build successfully compiles and runs in a development or production build mode as performed by the project team ("production build verified PASS" as reported).
- String-matching algorithms listed in the project materials have been exercised through the Smart Search interface ("string-matching algorithms tested" as reported).
- Dynamic programming and optimization modules were exercised ("DP and optimization modules confirmed" as reported).
- Flow and graph algorithms were exercised ("flow and graph algorithms confirmed" as reported).
- Routes/navigation and responsive UI were tested ("routes/navigation tested" and "responsive UI verified" as reported).
- Verification limits and constraints:
- The project materials do not include the execution environment versions, test logs, test counts, code coverage, benchmark protocols, or independent verification artifacts. Therefore, the report must not claim quantitative testing coverage or external validation beyond the project team’s statements.
- Screenshots showing selected UI states are demonstrative evidence of interface functionality for particular examples; the report will treat those values as screen-specific illustrative values rather than representative performance metrics.
- When the project materials list algorithm names, the report will refer to them as algorithms in the project materials list. It will not assert that every listed algorithm is proven to be present in the codebase without supporting implementation artifacts provided.

1.8 Organization of the report

This report is organized as follows: Section 1 defines the problem, objectives, scope, stakeholders and requirements; Section 2 presents related work, system design and algorithmic foundations; Section 3 records reported verification and proposes a reproducible evaluation plan; Section 4 interprets the supplied results and screenshots; and Section 5 concludes with a roadmap. The appendices include reference pseudocode, a cautious workflow guide, test checklist and glossary.

2. METHODOLOGY AND SYSTEM DESIGN

2.1 RELATED WORK AND TECHNICAL CONTEXT

Table 2.1. Search-algorithm trade-offs in the project context.

Method	Core idea	Time complexity	Space	Use / trade-off
Naive	Compare pattern at every alignment	O(nm) worst case	O(1) auxiliary; no preprocessing	Simple baseline; repeated scans can be costly
KMP	Prefix/failure function avoids rescanning	O(n+m) worst case	O(m)	Deterministic linear search for one pattern
Rabin–Karp	Rolling hashes with direct verification	Expected near O(n+m); O(nm) worst case	O(1) for one pattern, plus hash sets for batches	Convenient for hash-based batches; collisions require verification
Z-algorithm	Prefix matches in P + separator + T	O(n+m)	O(n+m)	Linear-time pattern matching using a Z-array
Suffix array + LCP	Index sorted suffixes, then query index	Build often O(n log n) simple; query O(m log n) basic	O(n)	Preprocessing cost can pay off for repeated queries

This subsection situates the project within the algorithmic and system-design literature cited by the project materials and distinguishes established algorithmic theory from project-specific evidence. The project materials reference canonical algorithmic sources (for example, Cormen et al., Knuth–Morris–Pratt, Rabin–Karp, Gusfield, Manber & Myers, Bellman, Needleman & Wunsch, Smith & Waterman, Ford & Fulkerson, Edmonds & Karp, Hopcroft & Karp) alongside practical online documentation (CP-Algorithms, GeeksforGeeks). The following subsections review these algorithm families and outline their typical strengths and limitations as relevant to an integrated document analytics system. Where the project materials list algorithms, the discussion treats them as background techniques the project intends to support; the report does not assert implementation completeness beyond "the project materials list" or the project team's own verification statements.

2.1.1 String-search and pattern-matching algorithms: families and trade-offs

String-search is foundational to the Smart Search portion of the project. The project materials enumerate multiple classical algorithm families: the Naive algorithm, Knuth–Morris–Pratt (KMP), Rabin–Karp, Z-algorithm, and suffix-based structures (suffix arrays and longest common prefix arrays). These families differ in preprocessing cost, worst-case and average-case runtime, and memory usage:

- Naive algorithm: conceptually the simplest approach, it examines every possible alignment of a pattern within a text. It is straightforward to implement and requires no preprocessing, but its worst-case runtime is O(nm) for text length n and pattern length m, which can be prohibitive on large documents.
- Knuth–Morris–Pratt (KMP): KMP improves worst-case performance by preprocessing the pattern to compute a failure function (also called the "prefix function" or "lps"). This preprocessing is O(m) and allows searching in O(n) time. KMP is robust in adversarial worst-case inputs because it guarantees linear time; however, KMP is tied to single-pattern matching and does not directly address multiple patterns without additional structures.
- Rabin–Karp: Rabin–Karp uses rolling hash values to compare pattern and substring fingerprints. It has average-case O(n + m) behavior under reasonable hashing assumptions and excels at multiple-pattern detection when several pattern hashes are maintained. Its worst-case behavior can degrade due to hash collisions unless additional checks are used. It lends itself to probabilistic performance trade-offs.
- Z-algorithm: The Z-algorithm computes the Z-array for a string (often formed as pattern + delimiter + text) and can perform linear-time pattern matching via Z-values. Like KMP, it guarantees linear-time behavior and is relatively simple to implement; the algorithm is particularly useful when multiple pattern attributes (prefix matches) are needed.
- Suffix arrays and LCP arrays: Suffix arrays are powerful for repeated searches across the same corpus because they index all suffixes of the text, enabling binary-search-based pattern matching in O(m log n) time plus an O(n) preprocessing cost to build the array (or O(n) with advanced algorithms). Coupled with LCP arrays, they support efficient repeated queries and can be adapted to multi-pattern tasks and substring queries. The trade-off is upfront preprocessing and memory for large documents or corpora.

The project materials' literature slide compares these methods with brief strengths and limitations. For an integrated system where users may choose different algorithms interactively, the platform must present these trade-offs to users and, where possible, supply appropriate preprocessing (for example, building suffix arrays) or fall back to linear-time algorithms for ad-hoc queries.

2.1.2 Sequence comparison and alignment algorithms

Document comparison in the project incorporates classical sequence-comparison techniques. The project materials list edit distance and the Wagner–Fischer algorithm, Needleman–Wunsch for global alignment, Smith–Waterman for local alignment, and longest common subsequence (LCS) methods. These algorithms are rooted in dynamic programming and have well-understood properties:

- Edit distance (Levenshtein distance) and Wagner–Fischer: computes the minimum number of single-character edit operations (insertions, deletions, substitutions) to transform one string into another. The Wagner–Fischer dynamic programming algorithm yields an O(nm) time and O(nm) space solution (where n and m are the string lengths), with standard optimizations to reduce memory usage to linear space when only distance (not full alignment) is required.
- Needleman–Wunsch (global alignment): originally developed for biological sequence alignment, this algorithm computes an optimal global alignment between two sequences using dynamic programming and a scoring function. It assumes an alignment covering entire sequences and is appropriate where end-to-end alignment is required.
- Smith–Waterman (local alignment): computes the best local alignment between substrings of two sequences by applying a modified dynamic programming recurrence that allows alignment to start and end at arbitrary positions; it is useful when searching for similar regions within larger sequences.
- Longest Common Subsequence (LCS): finds the longest sequence that appears as a subsequence (not necessarily contiguous) in both strings. The classic dynamic programming solution also runs in O(nm) time and space; LCS gives structural insight into shared content but is less sensitive to contiguous matches than alignment algorithms.

In a document-analytics context, these algorithms can be used to compare document drafts, detect plagiarism, identify localized edits, or produce visual diff-style alignments. The platform must offer clear parameterization (e.g., scoring schemes for alignment algorithms) and account for the computational cost of pairwise comparisons, especially for long documents.

2.1.3 Dynamic programming and optimization techniques

The Processing Optimizer in the project materials lists multiple optimization techniques that are classical dynamic programming problems: interval DP, knapsack DP, TSP, and matrix chain multiplication. These problems exemplify different DP patterns and applicability to workflow optimization:

- Interval dynamic programming: covers a family of problems where optimal solutions for intervals are composed from subintervals (examples include optimal matrix multiplication parenthesization and certain scheduling problems). Interval DP typically entails O(n^3) time for naive formulations, though problem-specific optimizations can reduce complexity.
- Knapsack DP: models resource allocation where items have weights and values and capacity constraints exist. The 0/1 knapsack has a classic pseudo-polynomial DP that runs in O(nW) time for capacity W and item count n.
- Traveling Salesman Problem (TSP): TSP in its general form is NP-hard; dynamic programming (Held–Karp) yields an O(n^2 2^n) exact algorithm suitable for small n. In practice, TSP formulations for workflow optimization are often approximated with heuristics or solved for small graphs.
- Matrix chain multiplication: classic DP to determine multiplication order that minimizes scalar multiplications; its algorithm runs in O(n^3) time and is a canonical example of DP on intervals.

The project's intent is to make these DP techniques available as building blocks for optimizing processing workflows that can be modeled as such combinatorial problems. The practicality of these methods depends on input sizes; the system must communicate computational trade-offs and may need to expose approximate or heuristic alternatives where inputs are large.

2.1.4 Flow and matching algorithms for network planning

The Network Planner module in the project materials lists Ford–Fulkerson, Edmonds–Karp, max-flow/min-cut theorems, bipartite matching, and assignment algorithms. These algorithms underpin resource-flow and allocation analyses:

- Ford–Fulkerson: a max-flow method that augments paths iteratively and is straightforward conceptually; its correctness follows from augmenting-path theorems. The basic method does not guarantee polynomial runtime unless augmenting paths are chosen carefully or capacities are integral.
- Edmonds–Karp: a specific implementation of Ford–Fulkerson that always selects the shortest augmenting path (in terms of number of edges) using BFS; it guarantees O(VE^2) time for capacities interpreted in certain ways.
- Max-flow/min-cut theorem: provides the theoretical equivalence between maximum flow and minimum cut capacity; it is central to interpreting flows as bottlenecks in resource networks.
- Bipartite matching and assignment algorithms: Hopcroft–Karp gives an efficient algorithm for maximum bipartite matching; assignment problems (Hungarian algorithm) solve minimum-cost perfect matching in bipartite graphs. These are applicable to assigning tasks to agents, scheduling, or pairing resources to demands.

In network planning for document-processing pipelines, these algorithms can model things such as maximum throughput under capacity constraints, assignment of processing tasks to agents or servers, or identification of bottlenecks. The choice of algorithm depends on graph size, integrality of capacities, and whether costs are part of the model.

2.1.5 Analytics, performance instrumentation, and the integrated pipeline

The project materials present an architecture that integrates document management, search, comparison, optimization, network planning, and analytics into a single platform. The pipeline (upload → read → tokenize → normalize → select

method → execute algorithm → generate result → visualize, with iterative improvement) reflects a data-processing workflow common in document analytics applications.

What makes an integrated system valuable, as articulated in the project materials, is the ability to connect these discrete algorithmic capabilities into workflows that permit end-to-end analysis: for example, a user can upload a set of documents, normalize text, run a pattern search to locate candidate regions, perform alignment to quantify differences, then model resource implications for processing variants via optimization or network planning, and finally view dashboard analytics that summarize performance and guide next steps. The integration matters because it reduces manual translation between separate tools, enables consistent tokenization and normalization across operations, and allows cross-module analytics (e.g., correlating search query characteristics with alignment costs or optimization outcomes).

The project materials list the Insights & Performance function as a first-class UI area: collecting analytics and performance measurements is therefore part of the platform's value proposition. However, the materials also set limits: verification statements are project-reported, and screenshots are illustrative. Therefore, claims about analytics accuracy or performance must be anchored to the specific data collected during a user session and not generalized.

2.1.6 Project-vs-background distinction and contributions

The project situates itself at the intersection of well-established algorithmic theory and an integrated front-end implementation that enables exploratory, interactive document analytics. The distinction between background (algorithmic theory, prior literature) and the project deliverable is crucial and must be maintained:

- Background: The algorithms and theoretical foundations referenced in the project materials (e.g., KMP, Rabin–Karp, suffix arrays, Wagner–Fischer, Needleman–Wunsch, Smith–Waterman, dynamic programming families, Ford–Fulkerson, Edmonds–Karp, Hopcroft–Karp) are established results with well-documented properties in the literature. The project materials cite canonical sources such as Cormen et al., Knuth et al., Rabin & Karp, Gusfield, Manber & Myers, Bellman, Needleman & Wunsch, Ford & Fulkerson, Edmonds & Karp, and others. These references supply the theoretical background and algorithmic templates on which the project builds.
- Project contribution: The primary project contribution, as described in the materials, is the engineering integration of these algorithmic techniques into a cohesive, browser-based platform that allows users to execute, compare, and visualize the results of search, comparison, optimization, and network algorithms within a single pipeline and interface. The implementation slide describes six user-facing functions: Documents, Smart Search, Document Compare, Processing Optimizer, Network Planner, Insights & Performance. Screenshots supplied are evidence that the UI supports particular states (for instance, a Smart Search screen with a selected Z-algorithm and a Document Compare & Alignment screen with options for edit-distance/LCS/global/local alignment). The project materials also report that these modules were tested or confirmed by the project team and that the responsive UI was verified.

This distinction matters for interpreting the deliverables: theoretical algorithms are background; the project's deliverable is the integration of these algorithms into interactive workflows, parameterizable UI components, and analytics instrumentation. The report must therefore avoid conflating well-known algorithmic guarantees from the literature with empirical performance claims for the delivered system unless those claims are supported by test logs or benchmarks, which were not provided.

Part 1 has defined the project problem, objectives, scope, constraints, stakeholder needs, and explicit requirements, and has situated the project within its technical context. The recommended next steps for project documentation are (a) a design section that maps functional requirements to specific components and data flows (for example, explicit data structures for tokenization and normalization, algorithm-selection UI wireframes, and interfaces for result visualization), (b) an implementation section documenting code organization and the project materials' verification statements with any available artifacts, and (c) an evaluation section that establishes reproducible benchmark protocols and test logs to substantiate performance and correctness claims beyond the project-reported confirmations. Part 2 should preserve the evidence boundaries established here and avoid overstating implementation status beyond the project materials list and the project team's reported verifications.

3. METHODOLOGY AND SYSTEM DESIGN

This section documents the methodology and system design decisions for the Intelligent Multi-Pattern Search and Document Analytics Engine (hereafter "the system"). It explains the system pipeline and modules, functional and nonfunctional requirements, string-search and sequence-comparison algorithm families considered, algorithm-selection as a documented UI concept, optimization and flow techniques used in the planner, and practical considerations (complexity, browser constraints, and real-time measurement). Where facts or diagrams derive directly from the provided project slides, that provenance is explicitly stated; conventional algorithmic background is described as established theory, with bibliographic pointers given in the source slides. No implementation internals are invented beyond what the slides supply.

2.2 Overview of system pipeline and modular decomposition



Figure 2.1. Methodology pipeline and listed technology stack from the supplied project presentation (source: project slide 6).

2.2.1 Pipeline (high-level)

The authoritative project pipeline, as depicted on the source presentation slide, is a linear-iterative flow: upload → read → tokenize → normalize → select method → execute algorithm → generate result → visualize, with iterative improvement. The presentation divides the product into the following high-level modules (also listed on the architecture slide):

- Document Management: upload, preview, tokenization and normalization; the supplied materials do not specify a persistence architecture.
- Search Engine: single- and multi-pattern matching modules supporting multiple algorithm choices.
- Comparison Engine: sequence comparison and alignment (edit distance, LCS, global/local alignments).
- Optimization Engine: dynamic-programming-based workflow optimization (interval DP, knapsack-like modules, matrix-chain-like cost minimization).
- Network Planner: graph-based network/route planning and resource allocation using flow and matching algorithms.
- Analytics and Performance: collection and presentation of runtime metrics and visualization of results.

2.2.2 Dataflow among modules

- Upload stage (Browser File APIs are named on the technology slide): files are introduced into the browser workflow; the exact state/persistence model is not specified.
- Read/tokenize: text is split into tokens according to tokenization policy (3.3).
- Normalize: casing, whitespace, punctuation handling, and other normalization occur (3.4).
- Method selection: user selects search/compare/optimization/network method via UI (slides show selection controls).
- Execution: the chosen algorithm runs on the normalized data in the browser context.
- Result generation and visualization: matches, alignments, optimized schedules or flows are assembled into displayable structures for visualization; analytics are collected for performance measurement.

This pipeline is the authoritative workflow defined in the slides; implementation details about state persistence, background services, or server-side components were not supplied and therefore are not assumed.

2.3 Functional and nonfunctional requirements

2.3.1 Functional requirements (derived from slides and the project objective)

- FR1: Ingest a variety of document types readable by browser File APIs and provide an in-browser preview.
- FR2: Tokenize and normalize text to produce inputs for search and comparison modules.
- FR3: Support single-pattern search and multi-pattern search with selectable algorithms (Naive, KMP, Rabin–Karp, Z, suffix-array/LCP).

- FR4: Provide document comparison functionality: edit distance (Levenshtein/Wagner–Fischer), LCS, global alignment (Needleman–Wunsch), local alignment (Smith–Waterman).
- FR5: Offer optimization tools: interval DP problems, knapsack DP formulations, TSP-like route optimization, matrix-chain multiplication-like cost minimization.
- FR6: Provide network planning algorithms: max-flow/min-cut, Ford–Fulkerson / Edmonds–Karp variants, bipartite matching and assignment procedures.
- FR7: Expose algorithm selection and configuration to the user as an explicit UI decision.
- FR8: Present analytics and performance metrics for user inspection.
- FR9: Allow iterative improvement: re-run with different methods or parameters and visualize differences.

2.3.2 Nonfunctional requirements (high-level constraints and aims)

- NFR1: Browser-native operation (React + TypeScript, Vite, Tailwind CSS, Browser File APIs are listed in the slides). Algorithms must be implementable in client-side JavaScript/TypeScript.
- NFR2: Responsiveness: UI actions should remain responsive; long-running computations should not block the main UI thread.
- NFR3: Usability: algorithm selection and parameters should be discoverable and documented in the UI (slides show algorithm selection in Smart Search screen).
- NFR4: Transparency: displayed results and performance data should include provenance (e.g., which algorithm was used).
- NFR5: Accuracy: algorithmic implementations must produce correct outputs consistent with theoretical definitions; numerical instability sources (e.g., hashing collisions) should be documented to users.
- NFR6: Safety limits: due to browser memory/CPU constraints, input sizes where algorithms may not be practical should be bounded or require explicit user confirmation.

2.4 Text ingestion, tokenization, and normalization considerations

2.4.1 Ingestion and preview constraints

The slides list Browser File APIs and show a preview capability. Browser-based ingestion implies:

- A client-side reading flow can place memory pressure on the browser for large files; the project deck does not specify the storage model.
- Previews should be generated by streaming where possible (e.g., reading in chunks) to avoid full-file blocking.

2.4.2 Tokenization

Tokenization is a domain-sensitive operation. Design choices exposed to users or defined as defaults should include:

- Granularity: character-level (for raw pattern matching), token/word-level (split on whitespace/punctuation), n-grams (for fuzzy search), or sentence-level (for document comparison).
- Token boundaries: handle punctuation, Unicode categories, hyphenation, and contraction handling per selected locale.
- Multi-pattern inputs: whether patterns are multiple literal strings or token sequences; tokenization must be consistent between pattern and document.

2.4.3 Normalization

Normalization steps are important to ensure appropriate matching behavior:

- Case folding: lowercase conversion is common for case-insensitive matching; locale-sensitive behavior must be noted (e.g., Turkish dotted/dotless i issues).
- Unicode normalization: NFC vs. NFD can affect byte/character sequences; the system should normalize according to a chosen Unicode Normalization Form (documented in UI).
- Whitespace and punctuation: options to collapse multiple whitespace to single space, remove punctuation, or retain punctuation depending on algorithm.
- Diacritics stripping: optional for language-agnostic searches.
- Stopword removal and stemming/lemmatization: optional for semantic searches but not for exact or sequence-based algorithms.

2.4.4 Consistency and provenance

All normalization and tokenization choices must be recorded with the search operation so results are reproducible within the session and to help users interpret outcomes. Slides show a pipeline that incorporates these steps; this is the authoritative workflow claim.

2.5 Single-pattern and multi-pattern exact search: problem framing

2.5.1 Problem definitions

- Single-pattern exact search: given text T of length n and pattern P of length m, find all occurrences of P in T.
- Multi-pattern exact search: given text T and a set of patterns S = {P1,...,Pk} with lengths m1...mk, find occurrences of any Pi in T. Complexity measures include total pattern length M = sum(mi) and maximum pattern length m_max.

2.5.2 Search desiderata

- Time complexity: ideally linear in n plus preprocessing cost in m.
- Space complexity: needs to be controlled due to browser memory constraints.
- Worst-case guarantees: some algorithms provide linear worst-case behavior (e.g., KMP); others are average-case linear (e.g., Rabin–Karp under hashing assumptions).
- Multi-pattern scalability: ability to search for many patterns efficiently (Aho–Corasick would be a classical approach, not listed on the slides; we therefore discuss multiple-pattern strategies compatible with the listed algorithms).

2.6 Exact string-matching algorithms — theory and characteristics

Table 2.2. Complexity summary (theoretical, not benchmarked in this project report).

Algorithm	Parameters	Time complexity	Space	Qualification
Naive search	Text n, pattern m	O(nm) worst case	O(1)	Single pattern
KMP	Text n, pattern m	O(n+m)	O(m)	Single pattern
Z search	Text n, pattern m	O(n+m)	O(n+m)	Single pattern
Rabin–Karp	Text n, pattern m	Expected O(n+m); worst O(nm)	O(1) plus hash set if batching	Hash collisions verified explicitly
Suffix array / LCP	Text n, pattern m	Build O(n log n) typical simple construction; basic query O(m log n)	O(n)	Repeated queries benefit from indexing
Edit distance / LCS	Sequences n and m	O(nm)	O(nm); distance-only can use O(min(n,m))	Traceback generally retains more state
Needleman–Wunsch / Smith–Waterman	Sequences n and m	O(nm) basic	O(nm) basic	Scoring scheme changes alignment, not basic order

This subsection summarizes algorithmic principles and complexity. Complexity statements explicitly indicate input-parameter dependence and worst-case qualifiers.

2.6.1 Naive (brute-force) matching

- Idea: for each alignment i in 0..n-m, compare P[0..m-1] with T[i..i+m-1] character by character until mismatch or full match.
- Time complexity: worst-case O(n·m) comparisons; for example with T = a^n and P = a^{m-1}b, this bound is tight.
- Space complexity: O(1) extra space beyond inputs.
- Use-case: small patterns or when simplicity and minimal preprocessing overhead are preferable.

2.6.2 Knuth–Morris–Pratt (KMP)

- Idea: preprocess pattern P to compute a longest proper-prefix-suffix (failure) function that allows skipping alignments upon a mismatch, enabling a linear-time search by reusing previous match information.
- Preprocessing time: O(m).
- Search time: O(n) worst-case.
- Space: O(m) for the failure table.
- KMP guarantees linear worst-case time performance independent of input content; suitable where deterministic guarantees are required.

The prefix-function treatment follows the classical KMP reference [2].

2.6.3 Rabin–Karp

- Idea: use rolling hash to compute fingerprints of substrings of text of length m and compare fingerprints against the pattern fingerprint; when hashes equal, perform explicit verification to avoid false positives.
- Average-case time: O(n + m) under ideal random hashing assumptions for single pattern; expected O(n + M) for multiple patterns if hashing supports multiple patterns.
- Worst-case time: O(n·m) if adversarial inputs cause many hash collisions; collisions may be theoretically rare for good hash schemes but must be explicitly documented.
- Space: O(1) extra for simple rolling hash; extended hashing structures may require more.

- Use-case: efficient average-case detection with low preprocessing, and useful when many patterns can be hashed into buckets (with careful collision control and explicit verification).

The rolling-hash method is attributed in the project references to Rabin and Karp [3].

2.6.4 Z-algorithm

- Idea: compute Z-array for a string S (often constructed as P + '$' + T) where Z[i] is the length of the longest substring starting at i that matches the prefix of S; pattern matches correspond to positions i where Z[i] ≥ m.
- Preprocessing/search: build Z in O(n + m) time for concatenated string.
- Space: O(n + m) for the combined string and Z-array.
- Worst-case time: O(n + m).
- Use-case: effective for single pattern searches and problems requiring prefix-based information.

2.6.5 Suffix arrays and LCP

- Idea: build a suffix array SA of all suffixes of T sorted lexicographically, with an LCP (longest common prefix) array to accelerate substring queries. Pattern matching reduces to binary search on SA.
- Preprocessing: suffix array plus LCP construction typically O(n) (using sophisticated algorithms) or O(n log n) with simpler approaches; construction is nontrivial and has memory costs.
- Query time: O(m log n) with binary search on SA plus O(m) character comparisons, or O(m + log n) with enhanced structures and LCP-guided searches.
- Space: O(n) for SA and LCP arrays; constant factors can be large and relevant in browser environments.
- Use-case: excellent for many repeated queries against static text because preprocessing cost is amortized across queries; suitable for multi-pattern workloads if many different patterns are queried against the same text.

Background references include Manber and Myers [5] and Gusfield [4].

2.6.6 Complexity summary and browser implications

- For short-lived, ad-hoc searches: KMP or Z provide linear worst-case guarantees with low overhead.
- For average-case speed with possible rare worst-case behavior: Rabin–Karp is often acceptable when collision probability is low and verification is used.
- For many repeated queries vs. static corpus: suffix arrays/LCP amortize preprocessing costs but trade larger memory footprints and construction cost.
- Naive search is simple and useful where pattern lengths are small relative to text or when development simplicity matters.

These algorithm descriptions reflect conventional algorithmic background; the slides include a literature-review comparison of these methods and list them as implemented/tested. The slide screenshots that show a selected Z-algorithm and an example query are UI evidence of algorithm-selectability at runtime and should be interpreted as illustrative, not as a measured benchmark. The slides report that "string-matching algorithms tested"; no formal benchmark data or measurement artifacts were supplied.

2.7 Multi-pattern search strategies and related complexity

2.7.1 Direct repeated single-pattern searches

- Strategy: run single-pattern algorithm (KMP/Rabin–Karp/Z) for each Pi in S sequentially. Complexity: sum over i of cost(T, Pi). For KMP this is O(k·n + M) because each KMP run is O(n + mi).
- Pros/cons: simple; redundant scanning of text for large k can be expensive.

2.7.2 Collective approaches

- Aho–Corasick is the canonical multi-pattern algorithm that constructs a trie of patterns plus failure links to search all patterns in O(n + M + output) time. Aho–Corasick is not listed on the project slides; therefore if implemented it must be documented as an extra choice. Given the listed algorithms, suffix-array approaches or hashing collections (Rabin–Karp with hashing of many patterns into buckets) are practical variants.
- Using a suffix array/LCP on T allows many pattern searches against the same T with individual pattern search cost O(m log n) as above; total cost for k patterns is O(sum(mi log n)) plus the O(n) or O(n log n) preprocessing.

2.7.3 Practical selection criteria (to be surfaced in UI)

The UI should present documented trade-offs rather than hidden defaults:

- Small k, large m: per-pattern KMP/Z might be simplest.
- Large k, static T: build suffix array/LCP for amortized queries.
- Many short patterns: Rabin–Karp with multi-hash table or Aho–Corasick (if available) may be best.

- Memory-limited environments: avoid suffix arrays if n is large; prefer streaming-friendly algorithms with O(1) extra space.

2.8 Sequence comparison and alignment: problem framing and algorithms

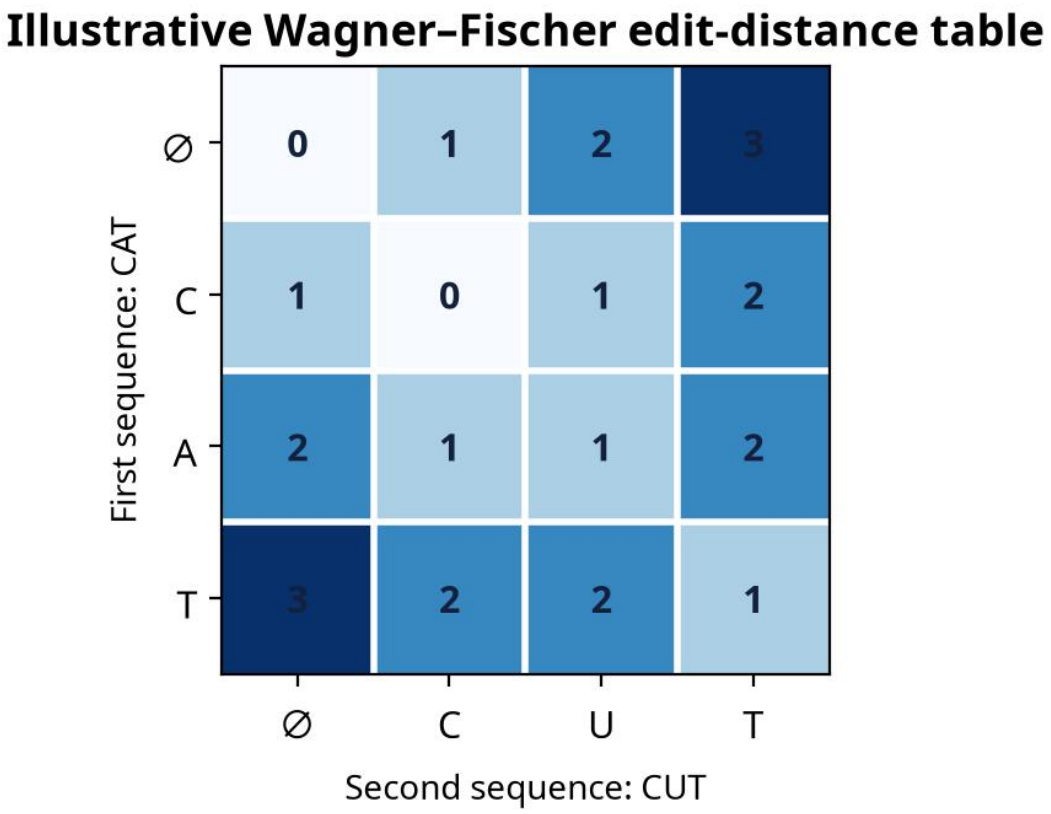


Figure 2.3. Illustrative Wagner–Fischer dynamic-programming table for CAT and CUT; this explanatory example is not a project benchmark.

Table 2.3. Comparison methods and interpretation.

Method	Optimizes	Scope	Interpretation
Edit distance (Wagner–Fischer)	Minimum insert/delete/substitute count	Whole strings	Simple numeric difference; depends on unit-cost edit model
LCS	Longest shared subsequence	Shared ordered content, gaps allowed	Emphasizes retained sequence; not a contiguous substring
Needleman–Wunsch	Score-optimal global alignment	End-to-end alignment	Requires match/mismatch/gap scoring choices
Smith–Waterman	Score-optimal local alignment	Best matching local regions	Useful where only portions correspond

2.8.1 Problem classes

- Edit distance (Levenshtein): minimum number of insertions, deletions, and substitutions to transform one sequence into another. Typically computed by dynamic programming (Wagner–Fischer).
- Longest Common Subsequence (LCS): find the longest sequence that is a subsequence of both inputs; related to edit distance with unit-cost edits.
- Global alignment (Needleman–Wunsch): align two sequences over their entire length maximizing a scoring function that may include match/mismatch scores and gap penalties (useful for sequence comparisons that penalize indels).
- Local alignment (Smith–Waterman): find highest-scoring local alignment (subsequence pair) with gap penalties; useful for detecting highly similar regions.

2.8.2 Wagner–Fischer (Levenshtein) DP

- Recurrence: DP[i][j] = min(DP[i-1][j] + 1 (deletion), DP[i][j-1] + 1 (insertion), DP[i-1][j-1] + cost(substitute)).
- Complexity: O(n·m) time and O(n·m) space for naive DP where n and m are sequence lengths. Space can be reduced to O(min(n,m)) if only the distance is required (not the alignment path).
- Use-case: exact edit-distance computation; larger inputs may be constrained by quadratic memory/time.

2.8.3 LCS

- DP similar to Levenshtein but optimizing for length of common subsequence:

DP[i][j] = DP[i-1][j-1] + 1 if a[i] == b[j] else max(DP[i-1][j], DP[i][j-1]).

- Complexity: O(n·m) time and space; standard sequence-analysis approach.
- Relation to edit distance: with unit-cost insert/delete and substitution cost of 2, edit distance and LCS are related; precise transformations depend on cost model.

2.8.4 Needleman–Wunsch (global alignment)

- Recurrence extends DP with a scoring matrix and gap penalties (linear or affine).
- Complexity: O(n·m) time and space in basic formulation; affine gap penalties introduce additional states (often implemented with three matrices) but keep the same asymptotic O(n·m).
- Output: alignment matrix yields optimal global alignment; traceback recovers alignment.

2.8.5 Smith–Waterman (local alignment)

- Similar DP structure as Needleman–Wunsch but with DP entries clipped at zero to allow local alignments:

DP[i][j] = max(0, DP[i-1][j-1] + s(a[i],b[j]), DP[i-1][j] - gap, DP[i][j-1] - gap).

- Complexity: O(n·m) time and space; identify maximum entry(s) and perform traceback to obtain local alignments.

2.8.6 Complexity qualifications and browser implications

- Quadratic DP algorithms are well-known to be computationally heavy for long documents. For browser-based operation:
- Practical upper bounds should be communicated to users (e.g., for sequences of length beyond tens of thousands, quadratic time and memory become prohibitive).
- Space savings: if only distance or score is required, rolling-row optimization reduces memory to O(min(n,m)).
- Parallelization: web workers or off-main-thread execution can preserve UI responsiveness for long-running DP tasks.
- The presentation slides state that DP and comparison modules were confirmed in testing; no measured performance numbers or environment details were supplied. That verification is a binary confirmation of module functionality, not a performance benchmark.

2.9 Algorithmic optimizations and advanced DP techniques

Table 2.4. Optimization and network problem families listed in the project materials.

Family	Problem model	Typical theoretical bound	Key input
Interval DP	Split an interval into subintervals; matrix-chain is a canonical case	Often O(n^3) in simple formulations	Input size n and recurrence-specific costs
0/1 Knapsack DP	Select items under a capacity bound	O(nW), pseudo-polynomial	Item count n; capacity W
TSP dynamic programming	Exact subset-state method (Held–Karp)	O(n^2·2^n) time; O(n·2^n) space	Suitable only for small n
Matrix-chain multiplication	Choose parenthesization minimizing scalar products	O(n^3) time; O(n^2) space	Matrix dimensions determine cost
Maximum flow / min cut	Route flow through capacity-constrained network	Edmonds–Karp O(VE^2)	Vertices V and edges E
Bipartite matching	Pair left/right sets without shared endpoints	Hopcroft–Karp O(E√V)	Graph size; matching structure
Assignment	Minimum-cost pairing across two sets	Hungarian O(n^3) for square n×n	Cost matrix and feasibility conditions

This subsection summarizes optimization topics listed on the slides (interval DP, knapsack DP, TSP, matrix chain) along with relevant algorithmic improvements and complexity qualifications.

2.9.1 Interval dynamic programming

- Problem family: optimization on intervals or contiguous substructures — examples include optimal matrix chain parenthesization or optimal binary search tree.
- Typical DP structure: DP[i][j] computed from partition points k in [i,j) with recurrence DP[i][j] = min_{k} (DP[i][k] + DP[k+1][j] + cost(i,k,j)).
- Complexity: naïve implementations are often O(n^3) time for n intervals; however, when quadrangle inequalities or monotonicity conditions hold, optimizations (Knuth optimization, divide-and-conquer DP) can reduce time to O(n^2) or O(n log n) depending on additional structure.
- Use-case: matrix-chain multiplication is a canonical interval DP.

2.9.2 Knapsack dynamic programming

- 0/1 knapsack DP: DP[w] = maximum value achievable with weight bound w yields O(n·W) time where W is capacity; pseudo-polynomial complexity.
- Optimizations: space reduction to O(W) by iterating items; meet-in-the-middle or approximation schemes (FPTAS) when W or values are large.
- Use-case in process scheduling: capacity-constrained resource allocation mapping to knapsack formulations.

2.9.3 Travelling Salesman Problem (TSP)

- TSP exact dynamic programming (Held–Karp): DP[mask][v] representing best cost to reach v after visiting nodes in mask; time O(n^2 · 2^n) and space O(n · 2^n).
- Approximations and heuristics: Christofides for metric TSP (3/2-approximation), nearest-neighbor, branch-and-bound, and metaheuristics are standard.
- Use-case: small to medium-sized routing problems may use dynamic programming; larger instances use heuristics.

2.9.4 Matrix chain multiplication

- Recurrence: DP[i][j] = min_{i ≤ k < j} (DP[i][k] + DP[k+1][j] + cost of multiplying resultant matrices).
- Complexity: naive O(n^3) time and O(n^2) space.
- Optimizations: similar monotonicity/Knuth optimizations apply when cost functions permit.

2.9.5 Practical recommendation and UI exposure

- The UI should allow users to select problem type and parameterize approximations (e.g., approximate TSP vs exact Held–Karp) and should warn when worst-case resources may be exceeded.
- Slides indicate the optimization modules were confirmed; they did not supply measured runtimes or input-size thresholds. Such thresholds must be set conservatively in the UI with explanatory notes.

2.10 Network flow, min-cut, and bipartite matching

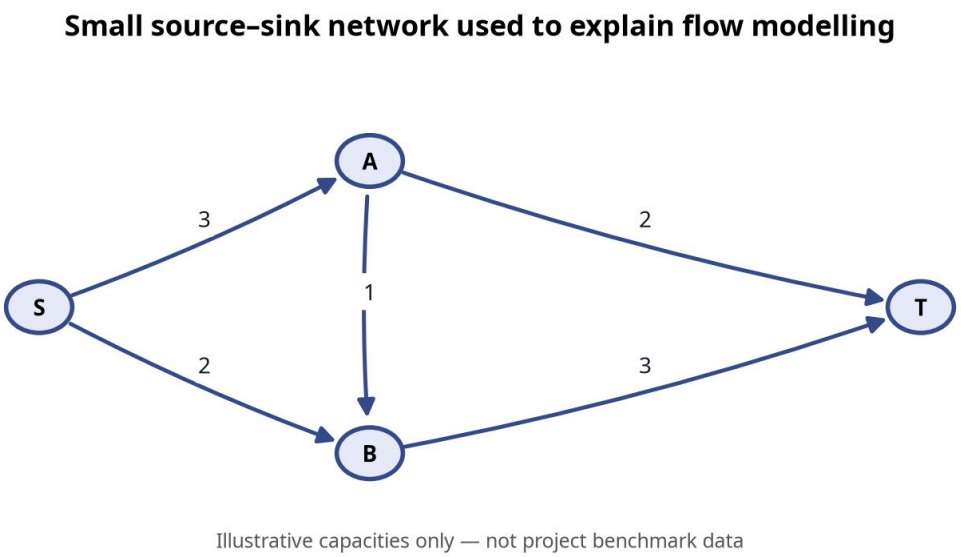


Figure 2.4. Illustrative capacitated network for explaining flow concepts; capacities are teaching examples, not project results.

2.10.1 Core problems and formulations

- Maximum flow: given a capacitated directed graph with source s and sink t, find the maximum feasible flow from s to t subject to capacity and conservation constraints.
- Min-cut: the dual problem — find a cut (partition of vertices) of minimum total capacity separating s and t; the max-flow min-cut theorem ties max flow value to min cut capacity.
- Bipartite matching: maximum matching in a bipartite graph; can be cast as a flow problem by creating source/sink nodes and unit-capacity edges.
- Assignment: weighted bipartite matching (Hungarian algorithm) finds minimum-cost perfect matching in O(n^3) time for n × n cost matrices.

2.10.2 Algorithmic approaches (as listed on slides)

- Ford–Fulkerson: augmenting-path method; runtime depends on augmenting path choice and capacities; for integral capacities and naive DFS augmentations, runtime is O(E · max_flow) which may be poor.
- Edmonds–Karp: a Ford–Fulkerson variant using BFS to find shortest augmenting paths in terms of edges, with guaranteed runtime O(V · E^2).
- Push–relabel and Dinic's algorithm, while not listed, are standard faster alternatives for some graphs; slides reference Edmonds–Karp explicitly.
- Hopcroft–Karp: specialized maximum matching algorithm for bipartite graphs with runtime O(sqrt(V)·E) and is referenced in the slides' literature list.

2.10.3 Complexity and input parameters

- Let V be number of vertices and E number of edges.
- Ford–Fulkerson (with unit capacities and BFS/DFS depending) has runtime dependent on max_flow F; worst-case O(E·F).
- Edmonds–Karp guarantees O(V·E^2).
- Dinic's algorithm: O(E·V^0.5) for unit networks or O(E·V^2/3) for general graphs in some analyses (not listed on slides; if referenced, must cite properly).
- Bipartite matching via Hopcroft–Karp: O(sqrt(V)·E).
- Assignment (Hungarian): O(n^3) for n × n assignment.

2.10.4 Use-cases and integration points

- Network Planner in the architecture slide is intended to use these algorithms for route/resource planning and for assignment problems.
- In-browser implementation caveats: graphs with very large V and E may exceed browser memory/time budgets. For graphs that model real-world network planning with moderate sizes, algorithms like Edmonds–Karp or Hopcroft–Karp are practical. The slides assert that flow and graph algorithms were confirmed; that confirmation is functional testing, not a statement about scale.

2.11 Algorithm selection as a documented UI concept

2.11.1 Motivation

The slides show UI screens that allow algorithm selection (e.g., Smart Search with Z-algorithm selected). The design intent is to make algorithm choice a transparent, discoverable action rather than an implicit internal decision. Presenting algorithmic options empowers users to make trade-offs based on understanding of their data and resource budget.

2.11.2 UI design principles for algorithm selection

- Expose algorithm names and short descriptions: show algorithm class (e.g., exact match, linear-time, rolling-hash average-case) and expected asymptotic behavior (e.g., O(n + m) worst-case).
- Parameter controls: allow thresholds (e.g., maximum pattern length for choosing KMP vs. suffix-array), or enable automatic auto-select with an explicit "recommendation" and visible rationale.
- Warnings and cost estimates: when a user selects an algorithm with potentially high cost (quadratic DP, suffix-array construction for very long texts), the UI should warn, estimate time/space complexity relative to current input sizes, and optionally require confirmation.
- Reproducibility strings: display tokenization/normalization choices and algorithm version used so results can be reproduced within the session.
- Default behavior: if "Auto" is selected, report which algorithm was chosen and why (e.g., "KMP selected because pattern length < X and text length < Y").

2.11.3 Exposing probabilistic guarantees and collision risks

For randomized or hashing-based methods (e.g., Rabin–Karp), the UI must clarify:

- Expected behavior (average-case time).
- Risk of hash collisions and the verification step performed to prevent false positives.
- Option to force deterministic alternatives (e.g., KMP) if the user requires guaranteed worst-case behavior.

2.11.4 Documentation provenance

The slides demonstrate selection controls and example UI states. Presenting algorithm selection in the UI is a documented project design requirement from the slides; the above UI principles are design recommendations consistent with the project objective to integrate method selection into the workflow.

2.12 Optimization of algorithmic performance and practical engineering constraints

2.12.1 Complexity-related optimizations

- Preprocessing amortization: expensive preprocessing (suffix arrays, FM-index building) can be justified if many queries run over the same text.
- Rolling memory techniques: for DP-based comparisons, if only scalar outputs are required, maintain only current and previous rows to reduce space to O(min(n,m)).
- Early termination: in pattern matching, allow early exit when matches are found according to policy (e.g., stop after k hits), to bound runtime.
- Index-based heuristics: maintain inverted indices or n-gram indices for faster multi-pattern or approximate matching (these are implementation-level indexing choices and would be documented if used).

2.12.2 Parallelism and asynchrony in the browser

- Web workers: offload CPU-intensive algorithms to separate threads (web workers) to keep the main thread responsive.
- Chunked processing: process large texts in chunks and stream results to the UI incrementally to improve perceived responsiveness and allow user cancellation.
- Incremental algorithms: algorithms that can generate partial results (e.g., suffix-tree traversal that yields matches) are preferable for interactive contexts.

2.12.3 Memory constraints and data representation

- Character vs. code-unit representation: JavaScript uses UTF-16 code units; string length and indexing semantics differ from codepoint counts. For correct Unicode handling, normalization and codepoint-aware iteration are necessary.
- Avoid materializing large auxiliary arrays where possible. For DP tables, consider typed arrays only if memory and performance demand it; but their use increases code complexity and must be justified.

2.12.4 Measurement and instrumentation

- Collect runtime and memory metrics per operation (wall-clock time, peak memory usage approximations) where permitted by the runtime.
- Present results with clear provenance and caveats: the slides include a “Insights & Performance” UI and state that performance checks were “verified PASS.” No benchmark protocols, environment versions, or reproducible numeric measurements were provided; therefore any shown numbers in the UI must be screen-specific illustrative values or derived from ad-hoc runs and must be labeled as such.

2.13 Real-time measurement, browser-side constraints, and UX considerations

2.13.1 Real-time measurement limitations

- Browser environments provide limited and platform-dependent facilities for precise performance measurement. Typical practice uses performance.now() for high-resolution timing; memory measurement is coarse and platform-dependent.
- The slides list “Insights & Performance” and confirm responsive UI tests. However, no formal benchmarking artifacts were supplied; therefore any in-app real-time metrics must be presented as best-effort diagnostics, not certified benchmarks.

2.13.2 Main-thread blocking risks

- Synchronous heavy computations block UI rendering and may yield unresponsive pages. Use of web workers, requestAnimationFrame, or yielding heuristics is necessary to keep the UI responsive.
- The architecture and implementation slides indicate a responsive UI was verified; this is functional testing evidence. The design must implement asynchronous execution patterns for long-running algorithms.

2.13.3 Resource caps and user controls

- Implement user-configurable caps on runtime, memory, or processed input sizes. For example: maximum text length for DP alignment tasks; maximum number of patterns for naive per-pattern scans.
- Provide progress indicators and cancellation controls for long-running operations.

2.13.4 Security and privacy considerations

- Files are read locally via browser File APIs; avoid transmitting user documents without explicit action.
- Sanitize any displayed contents to avoid embedding active content.

2.14 Provenance, testing claims, and limitations

2.14.1 What is asserted directly from the slides

- Project title, team, department and college are authoritative facts from the supplied presentation.
- The pipeline upload → read → tokenize → normalize → select method → execute algorithm → generate result → visualize, and the architecture and module decomposition (document management, search engine, comparison engine, optimization engine, network planner, analytics) are depicted in the slides and are taken as design requirements.
- The technology stack components listed on the slides (React + TypeScript, Vite, Tailwind CSS, Lucide React, React Router, Browser File APIs; modern browser, Node.js, npm) are accepted as environment targets; versions and backend/persistence details were not supplied and are explicitly not assumed.
- The slides list the algorithms and techniques implemented/tested: search—Naive, KMP, Rabin–Karp, Z-algorithm, suffix array/LCP; comparison—edit distance, Wagner–Fischer, Needleman–Wunsch, Smith–Waterman, LCS; optimization—interval DP, knapsack DP, TSP, matrix chain; network—Ford–Fulkerson, Edmonds–Karp, max-flow/min-cut, bipartite matching, assignment. The slides report modules "confirmed" or "tested" (string-matching algorithms tested; DP and optimization modules confirmed; flow and graph algorithms confirmed). These are functional confirmations, not measured performance claims.

2.14.2 What is conventional algorithmic background vs. slide-derived evidence

- The algorithmic descriptions, asymptotic complexities, and optimization techniques presented in Sections 3.5–3.9 are conventional algorithmic background. The slides cite canonical sources (Cormen et al., Knuth–Morris–Pratt, Rabin & Karp, Gusfield, Manber & Myers, Bellman, Needleman & Wunsch, Smith & Waterman, Ford & Fulkerson, Edmonds & Karp, Hopcroft & Karp; CP-Algorithms and GeeksforGeeks are listed as references). Those background details are well-established and are presented here to provide instructional clarity. If a precise citation in the slides lacks publication details, that incompleteness is noted in the references slide.
- The UI screenshots in the supplied slides (Smart Search with Z-algorithm selected; Document Compare & Alignment screen showing edit-distance/LCS/global/local alignment options) are evidence of UI states and confirm that algorithm selection is surfaced. Any numbers or sample outputs visible in screenshots are illustrative and must be treated as screen-specific examples rather than reproducible measurements.

2.14.3 Testing and verification transparency

- Slides report that a production build was verified PASS and that various algorithm modules were tested/confirmed. No test logs, environment versions, test counts, coverage data, benchmark procedures, or independent verification artifacts were supplied. Therefore:
- Functionality claims from slides are treated as developer-reported confirmations of module operation.
- No performance or scalability claims beyond these confirmations are asserted; any recommended input-size thresholds or resource caps in this report are conservative engineering suggestions, not measured limits from the project artifacts.

2.14.4 Documentation and traceability

- All decisions to expose algorithmic choices in the UI should be accompanied by textual explanations (in-app help screens, tooltips) documenting asymptotic complexities, assumptions (e.g., deterministic vs probabilistic behavior), and trade-offs.
- Any optional randomized algorithms (e.g., Rabin–Karp with probabilistic hashing) require documentation of the verification step and collision risk.

Concluding notes on design methodology

- The methodology centers on exposing algorithmic choices and provenance to users, implementing algorithms with explicit complexity awareness, and placing practical resource bounds in an interactive browser environment.
- The system design combines algorithmic rigor (use of classical linear-time string algorithms and DP-based comparison) with engineering practices suitable for client-side operation (asynchronous execution, memory-aware data structures, and user-visible algorithm selection).
- Implementation details beyond the slides (backends, persisted storage, or versioned environment details) are not assumed. The slides' reported functional confirmations are respected while their limitations as non-benchmarked assertions are explicitly acknowledged.

References and further reading (as provided by slides)

- Knuth, D., Morris III, J., & Pratt, V., "Fast Pattern Matching in Strings," SIAM J. Computing, 1977.
- Rabin, M. O., & Karp, R. M., "Fingerprinting by Random Polynomials," (technical report), 1981.
- Manber, U., & Myers, G., "Suffix Arrays: A New Method for On-Line String Searches," SIAM J. Computing, 1993.
- Gusfield, D., Algorithms on Strings, Trees, and Sequences, Cambridge University Press, 1997.

- Bellman, R., Dynamic Programming, Princeton University Press, 1957.
- Needleman, S. B., & Wunsch, C. D., "A General Method Applicable to the Search for Similarities in the Amino Acid Sequences of Two Proteins," Journal of Molecular Biology, 1970.
- Smith, T. F., & Waterman, M. S., (1981).
- Ford, L. R., & Fulkerson, D. R., Flows in Networks, Princeton University Press, 1962.
- Edmonds, J., & Karp, R. M., (1972).
- Hopcroft, J., & Karp, R., (1973).
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, 3rd ed., MIT Press, 2009.
- Additional sources referenced on the slides: Horowitz et al., Fundamentals of Computer Algorithms; CP-Algorithms; GeeksforGeeks.

(Where the slide's reference lacked full bibliographic details, the slide author's reference list is reproduced as provided. This report does not invent missing publication details.)

End of Part 2: Methodology and System Design.

3. EXPERIMENTAL DESIGN AND VERIFICATION

This part of the report separates (A) the verification statements that the project presentation explicitly reported and for which the report provides direct evidence, from (B) a reproducible experimental protocol that is proposed but was not performed. The objective is to document what the authors have verified, to enumerate missing evidence and uncertainties, and to provide a concrete, reproducible experimental plan that an independent party (or the project team in a subsequent iteration) can execute to validate functional correctness and measure performance.

3.1 Scope and verification objectives

Table 3.1. Project-reported verification and evidence limitations.

Area	Reported status	Source evidence	Missing for independent audit
Production build	PASS / verified successfully	Build outcome stated in source slide	No command output/build artifact supplied
String matching	Algorithms tested/verified	High-level test statement in slide	No test list, logs, counts or environment
DP and optimization	Modules confirmed	High-level confirmation in slide	No case-level outputs or coverage
Flow and matching	Confirmed	High-level confirmation in slide	No graph fixtures or expected/observed outputs
Routes/navigation	Verified	Navigation/routes tested in slide	No route matrix or automation logs
Responsive UI	Verified	Responsive UI tested in slide	No viewport/device matrix

- Verified items reported by the project (explicit statements appearing in the supplied presentation): production build PASS; string-matching algorithms tested/verified; dynamic programming (DP) optimization modules confirmed; flow and graph algorithms confirmed; routes/navigation tested; responsive UI verified. These items are treated below as "reported verifications" and are transcribed exactly as reported.
- Experimental goals (proposed, not performed): (a) verify functional correctness across document ingestion, tokenization/normalization, single- and multi-pattern search, comparison (edit distance, LCS, alignment), DP optimization, network/flow algorithms, and UI navigation; (b) produce repeatable performance microbenchmarks that measure execution time and memory across controlled inputs and algorithm choices; (c) measure responsiveness and interactivity of the UI under realistic workloads; (d) establish confidence bounds (median, 10th/90th percentiles) from repeated runs and warmup phases.

3.2 Verification vs. reproducible experiments — evidence boundaries

It is important to distinguish evidence that was reported by the project from measurements that have been neither supplied nor performed. The presentation supplies interface screenshots, a list of implemented algorithms and modules, and a clear production-build PASS statement. However, the presentation does not include environment versions (browser, Node.js, npm, OS), test logs, counts of tests executed or their pass/fail results, coverage statistics, raw benchmark data, or independent audit statements. Therefore:

- Reported verification statements (quoted wording preserved): "production build PASS"; "string-matching algorithms tested/verified"; "DP optimization confirmed"; "flow and matching confirmed"; "navigation and responsive UI tested".
- These are treated as project-reported verifications. The report reproduces them exactly as the team presented, and explicitly notes that no detailed test artifacts or raw benchmark datasets were supplied.
- All subsequent proposed benchmarks described in sections 4.6–4.10 are explicitly proposed, not executed by the project team, and therefore yield no empirical numbers in this report.

3.3 Test categories and high-level verification plan

The functional and performance areas to be tested fall into the following categories; each category lists a concise set of test objectives and acceptance criteria.

1. Document ingestion and preview

- Objectives: verify upload of local files via Browser File APIs, successful parsing (read stage), preview rendering, tokenization and normalization correctness.
- Acceptance criteria: given test documents, ingestion completes without unhandled exceptions; tokenization output matches expected token streams for canonical test cases; normalization (case-folding, punctuation handling, Unicode normalization) behaves as specified for examples.

2. Exact single-pattern search

- Objectives: verify correctness of Naive, KMP, Rabin–Karp, Z-algorithm, suffix-array/LCP-based exact searches on canonical examples including repeats and overlaps.

- Acceptance criteria: each algorithm returns identical location lists for the same input when algorithmic preconditions hold (e.g., hash base modulus choices when using Rabin–Karp are deterministic for test).

3. Multi-pattern search

- Objectives: verify that the engine can handle multiple patterns in a single query (either via repeated single-pattern search or an explicit multi-pattern routine) and that results correctly identify all occurrences for each pattern.
- Acceptance criteria: all patterns' occurrences are returned without omission; overlapping matches and duplicate pattern handling are consistent with the documented behavior.

4. Empty and boundary cases

- Objectives: test zero-length inputs (empty document, empty pattern), single-character inputs, maximal reasonable sizes for in-browser operation, and pathological inputs (e.g., repetitive sequences causing worst-case behavior for certain algorithms).
- Acceptance criteria: no unhandled exceptions; well-defined outputs (e.g., empty match set) and UI messages.

5. Document comparison, edit-distance, LCS, alignment

- Objectives: verify implementations of Wagner–Fischer (edit distance), Needleman–Wunsch (global alignment), Smith–Waterman (local alignment), and Longest Common Subsequence (LCS) produce correct distances/alignments for canonical test pairs.
- Acceptance criteria: computed numeric distances and alignments match reference results from standard textbooks or small hand-computed examples.

6. Dynamic programming (interval DP, knapsack, matrix chain)

- Objectives: verify correctness of DP optimization modules on canonical examples with known optimal values.
- Acceptance criteria: outputs match reference optimal values and (where applicable) reported optimal solution traces.

7. Flow and matching (Ford–Fulkerson / Edmonds–Karp / bipartite matching)

- Objectives: verify flow algorithms compute correct max-flow/min-cut on canonical networks and that matching/assignment routines succeed on standard bipartite examples.
- Acceptance criteria: numeric max-flow values and matching results equal reference solutions.

8. Navigation, responsiveness, and UI behavior

- Objectives: check that navigation routes, UI layout, and responsive behaviors operate correctly across viewport sizes and that interactive elements (search boxes, file upload, algorithm selection) behave without blocking or crashes.
- Acceptance criteria: UI state transitions occur as designed; examples shown in screenshots are reproducible as UI states.

3.4 Test harness and instrumentation (proposed)

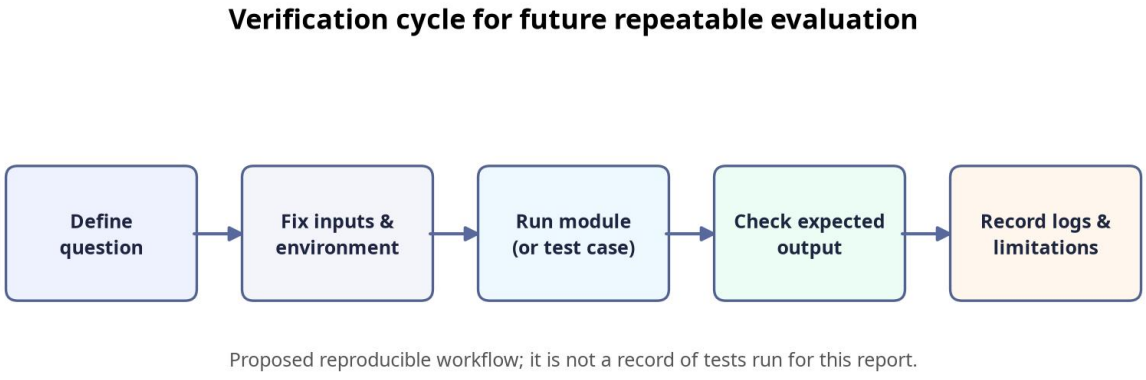


Figure 3.1. Proposed verification cycle. It describes recommended process steps and is not evidence of tests performed.

To make tests reproducible and to separate functional correctness from performance artifacts (JIT warmups, GC, system noise), the following harness is proposed.

- Environment recording: capture and store (a) exact browser user agent string, (b) operating system name and version, (c) Node/npm versions if build steps are part of the test workflow, and (d) CPU and memory capacity of the test host. This is a required part of any reproducible run.
- Deterministic test inputs: create a curated set of test documents and pattern lists. Each test artifact must be stored as a file with a content hash (e.g., SHA-256) recorded in the test manifest.

- Functional test runner: a scripted harness (e.g., headless browser automation via Playwright or Puppeteer) that performs upload, selection of algorithm, triggers execution, retrieves outputs, and asserts equality against reference outputs for functional tests.
- Instrumentation: measure wall-clock time and memory (heap size) using in-browser performance.now() and performance.memory (if available), plus collection of console logs and error traces. For algorithms implemented in TypeScript/JavaScript, collect counts of key algorithmic steps (e.g., character comparisons, table fills) if the implementation exposes instrumentation hooks; these counts must be deterministic or taken as approximate diagnostic metrics.
- Storage of raw logs: tests should save raw outputs, timestamps, console logs, and captured screenshots into a reproducible archive.

3.5 Controlled inputs and test matrices (proposed)

Table 3.2. Selected proposed functional test cases; execution is not claimed.

ID	Area	Input / scenario	Expected check
S-01	Exact match	“ababcababc” / “ababc”	Expect indices 0 and 5; compare outputs across available exact matchers
S-02	Overlapping matches	“aaaaa” / “aa”	Document overlap policy; if overlaps allowed, indices 0,1,2,3
S-03	Multi-pattern	“abcababc”; patterns “ab”, “abc”, “bca”	Expected: ab at 0,3,5; abc at 0,5; bca at 1
C-01	Edit distance	“kitten” / “sitting”	Unit-cost Levenshtein distance 3
C-02	LCS	“AGGTAB” / “GXTXAYB”	Length 4; one valid LCS is “GTAB”
O-01	0/1 knapsack	weights 2,3,4; values 3,4,5; capacity 5	Optimal value 7 using weights 2 and 3
N-01	Max flow	Small graph with declared capacities	Compare max-flow value with independent reference; verify conservation
UI-01	Navigation and search	Dashboard → Smart Search → choose method → run	Verify view transition, method label, result view; proposed automated case

Design test inputs to cover typical and worst-case behaviors across algorithms. Inputs must include explicit size and content constraints. The proposed matrix uses fixed-length buckets and explicit pattern counts:

- Text length buckets: 1 KB (small), 10 KB (medium), 100 KB (large), 1 MB (very large; to be used only if browser environment can handle in memory).
- Pattern sizes: 1 char, 3–10 chars (short), 20–100 chars (medium), pattern equal to text length (full match).
- Pattern counts (for multi-pattern): 1, 5, 20, 100.
- Pattern types: random alphabetic, repetitive (e.g., "aaaa..."), structured (words from text), and adversarial patterns constructed to induce worst-case behavior in vulnerable algorithms (e.g., Z or naive worst-case for specific inputs).
- Document types: plain English prose, code-like text with punctuation, Unicode text including combining marks, and binary-like data represented as text (to stress tokenization).

Each test is identified by a unique descriptor: TEXT{size}-{type}_PAT{count}-{patternType}_{alg}.

3.6 Warmup, repetition, and statistics (proposed)

Table 3.3. Proposed benchmark protocol; no benchmark results are included here.

Dimension	Proposed design	Reporting requirement
Input dimensions	Text 1 KB / 10 KB / 100 KB; optional 1 MB only if stable	Record actual bytes and token/code-unit length
Patterns	1, 5, 20, 100; short/medium/near-text-length	Same exact patterns for all comparable runs
Input families	Random, repetitive, structured, Unicode	Store input files and content hashes
Warm-up and repeats	Propose 10 warm-ups and 51 measured repeats where practical	Record reductions for expensive runs; do not silently drop outliers
Statistics	Median, 10th/90th percentile, min/max	Report units, sample size and host metadata
Environment	Browser/engine, OS, CPU, memory, Node/npm for build	No such metadata was included in supplied materials

To reduce measurement noise and account for runtime optimizations and garbage collection, the proposed benchmarking protocol uses warmup runs and repeated timing with statistical summarization.

- Warmup: perform W = 10 warmup executions for each algorithm-input pair that will not be recorded. Warmup runs help JIT compilation stabilize.

- Measured repeats: perform R = 51 recorded runs for each algorithm-input pair. The choice of an odd number facilitates median computation.
- Reporting: compute and report the median, 10th percentile, and 90th percentile of wall-clock times across recorded runs. Also report the minimum and maximum. For memory, report peak and median resident heap or memory as available.
- Outlier handling: if a single run is clearly contaminated (e.g., the event loop blocked by unrelated system activity) and is more than 5× the median, record it as outlier and report both inclusive and exclusive statistics.
- Time budget considerations: for large inputs that would make 51 runs prohibitively long, reduce R but record the reduction and justify it in the test log.

3.7 Functional correctness tests — specific examples and expected behaviors (proposed)

Table 3.2. Selected proposed functional test cases; execution is not claimed.

ID	Area	Input / scenario	Expected check
S-01	Exact match	“ababcababc” / “ababc”	Expect indices 0 and 5; compare outputs across available exact matchers
S-02	Overlapping matches	“aaaaa” / “aa”	Document overlap policy; if overlaps allowed, indices 0,1,2,3
S-03	Multi-pattern	“abcababc”; patterns “ab”, “abc”, “bca”	Expected: ab at 0,3,5; abc at 0,5; bca at 1
C-01	Edit distance	“kitten” / “sitting”	Unit-cost Levenshtein distance 3
C-02	LCS	“AGGTAB” / “GXTXAYB”	Length 4; one valid LCS is “GTAB”
O-01	0/1 knapsack	weights 2,3,4; values 3,4,5; capacity 5	Optimal value 7 using weights 2 and 3
N-01	Max flow	Small graph with declared capacities	Compare max-flow value with independent reference; verify conservation
UI-01	Navigation and search	Dashboard → Smart Search → choose method → run	Verify view transition, method label, result view; proposed automated case

The following are explicit functional correctness tests that any reproducible harness should execute. Numbered examples are concrete and deterministic; reference outputs must be stored.

1. Document ingestion: Upload a 5 KB UTF-8 text containing the sentence "The quick brown fox jumps over the lazy dog." Verify that tokenization (split on whitespace and punctuation) yields the token sequence ["The","quick","brown","fox","jumps","over","the","lazy","dog"]. Verify normalized tokens after lowercasing produce ["the","quick","brown","fox","jumps","over","the","lazy","dog"].
2. Empty inputs: Upload an empty file and search for an empty string and a non-empty pattern "a". Expected behavior: search for non-empty pattern returns zero matches; searching for empty pattern either returns a defined behavior (documented in the system) or returns an error/validation message. The harness must assert whichever documented behavior the implementation exposes.
3. Single exact match: Text "ababcababc", pattern "ababc". Expected match positions: index 0 and index 5 (0-based). Run Naive, KMP, Rabin–Karp (with deterministic hash) and Z-algorithm to confirm identical location lists.
4. Overlapping matches: Text "aaaaa", pattern "aa". Expected matches at indices 0, 1, 2, 3 (if overlaps are allowed). Verify that each search mode returns either overlapping matches or non-overlapping matches per the documented semantics.
5. Multi-pattern example: Text "abcababc", patterns ["ab","abc","bca"]. Expected results: pattern "ab" at positions 0, 3, 5; "abc" at 0, 5; "bca" at 1. Verify correct mapping from pattern to matches across the multi-pattern facility.
6. Rabin–Karp hash correctness: For small text and pattern, compute hash values by the implementation and compare with independently computed deterministic hash for the same base/modulus to ensure collisions are handled correctly (i.e., candidate verification by direct comparison).
7. Edit distance/Wagner–Fischer: Input strings "kitten" and "sitting". Expected edit distance: 3. Verify distance value and that the backtrace produces a valid alignment consistent with that distance.
8. Global alignment (Needleman–Wunsch): Using simple scoring (+1 match, −1 mismatch, −1 gap), align "GATTACA" and "GCATGCU". Expected global alignment score and an alignment path consistent with reference solutions in standard literature.
9. Local alignment (Smith–Waterman): With a small scoring matrix where a known local alignment exists, verify that the algorithm finds the highest-scoring local alignment and that the reported substring matches the reference.
10. LCS: Strings "AGGTAB" and "GXTXAYB". Expected LCS length 4 and one LCS "GTAB". Verify length and one returned subsequence.

11. DP optimization — knapsack: Given items with weights [2,3,4], values [3,4,5], capacity 5, expected optimal value 7 (select the first two items). Verify reported max value and chosen item indices.
12. Interval DP / matrix chain: For small matrix dimensions with known optimal parenthesization, verify the computed minimal cost and parenthesis structure.
13. Max-flow (Edmonds–Karp): On the canonical network with capacities producing max-flow value 23 (use a standard textbook example), verify computed max-flow equals the expected reference. Store flow assignments to edges and verify flow conservation constraints.
14. Bipartite matching: On a 3x3 bipartite graph with known maximum matching size 3, verify matching output covers expected pairs.
15. Navigation and UI behavior: Using automated UI tests, navigate from Dashboard → Smart Search → Select Z-algorithm → Execute query and verify the UI shows a results pane, that the selected algorithm indicator is present, and that the search input field and upload controls remain functional after the search completes.

Each functional test must assert equality (for deterministic tests) or membership (e.g., that the result is one of several acceptable canonical answers) against stored reference outputs. Where multiple valid alignments exist, the harness should validate score and consistency, not exact string alignment unless a single canonical representation is enforced.

3.8 Performance microbenchmarks — proposed protocol

Table 3.3. Proposed benchmark protocol; no benchmark results are included here.

Dimension	Proposed design	Reporting requirement
Input dimensions	Text 1 KB / 10 KB / 100 KB; optional 1 MB only if stable	Record actual bytes and token/code-unit length
Patterns	1, 5, 20, 100; short/medium/near-text-length	Same exact patterns for all comparable runs
Input families	Random, repetitive, structured, Unicode	Store input files and content hashes
Warm-up and repeats	Propose 10 warm-ups and 51 measured repeats where practical	Record reductions for expensive runs; do not silently drop outliers
Statistics	Median, 10th/90th percentile, min/max	Report units, sample size and host metadata
Environment	Browser/engine, OS, CPU, memory, Node/npm for build	No such metadata was included in supplied materials

To compare algorithmic performance (not an exhaustive profiling exercise), the following microbenchmark protocol is proposed:

- For each algorithm in the Search category (Naive, KMP, Rabin–Karp, Z, suffix-array based), and for each text/pattern pair in the controlled matrix (section 4.5), execute the warmup and repetition scheme (section 4.6).
- Record the median wall-clock time for algorithm completion and the median of any available internal counters (e.g., character comparisons).
- Compare algorithms by plotting (or reporting) median times on identical input pairs. For each report, explicitly report the environment metadata (browser, OS, CPU). Note: this report does not include such measurements because the project did not supply them; the protocol is provided so that future runs are reproducible.
- For memory-sensitive routines (e.g., suffix-array construction and DP tables), record peak heap memory during execution and flag tests where memory usage exceeds 50% of available process memory to avoid browser instability.

3.9 UI responsiveness testing — proposed approach

User-perceived responsiveness is a separate class of test and requires instrumentation at the UI level.

- Procedure: use a headless or automated browser to trigger user actions (file upload, algorithm selection, search execution) and measure time from action initiation to first meaningful paint of results and to completion of result rendering.
- Interactive responsiveness: measure time between user input and UI feedback when typing or navigating results (e.g., search-as-you-type). For long-running algorithms, verify that progress indicators are displayed and that input controls remain usable.
- Metric reporting: time to first result, time to full rendering, and a responsiveness index (fraction of test cases where first result is within a target threshold, e.g., 200 ms).
- Note: The presentation includes screenshots that show example UI states. These are snapshots; any timing values visible in screenshots (for example if any milliseconds were displayed) are UI-specific illustrative values, not general benchmark results. The report treats screenshots only as interface evidence, not as measured performance data.

3.10 Test artifacts, missing evidence, and validation gaps

The presentation reports testing/confirmation for several modules and states the high-level PASS/confirmation statements listed in section 4.2. However, crucial artifacts for independent verification are missing from the supplied materials:

- No environment/version metadata (browser name/version, Node.js/npm versions, OS, CPU) were provided.

- No test logs or structured test reports listing test cases executed, pass/fail counts, or failure traces were supplied.
- No code coverage percentages or unit/integration test coverage reports were supplied.
- No raw benchmark timing data, run histories, or measured distributions were supplied.
- No independent audit or third-party verification artifacts were supplied.
- No build artifacts or reproducible container images for exact builds were included.

Because of these missing artifacts, this report cannot independently reproduce or validate the project's reported verifications. The proposed experimental protocol above is intended to close these gaps in future work.

4. RESULTS AND DISCUSSION

This section summarizes the verified statements from the project presentation, discusses what the presented evidence supports and does not support, and interprets possible conclusions while avoiding any fabrication of performance numbers. The section is divided into discrete discussions of correctness, algorithmic validation, optimization modules, flow/matching, UI and navigation, testing completeness, limitations, and recommended next steps.

4.1 Exact transcription of verified result statements

Table 4.1. Evidence summary by project module.

Module	Stated capability	Evidence available	Limit
Documents	Upload, preview, tokenization/normalization pipeline	Module is described; screenshots demonstrate UI context	Broader PDF/DOCX support and persistence are roadmap items
Smart Search	Single/multi-pattern search, algorithm choice, highlighting	Search screen shows Z-algorithm selection and example results	One screenshot is not a benchmark or exhaustive test log
Document Compare	Edit distance, LCS, global/local alignment	Compare screen shows methods and sample outputs	Displayed numbers are screen-specific examples
Processing Optimizer	DP-based workflow/interval/resource optimization	Module and algorithm family listed; DP confirmed by project	No detailed test cases or raw outputs supplied
Network Planner	Flow, min-cut, matching, assignment	Module and algorithm family listed; flow/matching confirmed	No test graph, logs or scale evidence supplied
Insights & Performance	Analytics and performance measurements	Dashboard/analytics screen shown	No reproducible benchmark protocol or raw run data supplied

The presentation reports the following verification statements; these are transcribed exactly as reported by the authors:

- production build PASS
- string-matching algorithms tested/verified
- DP optimization confirmed
- flow and matching confirmed
- navigation and responsive UI tested

These statements are repeated here exactly as the project presented them; the report does not expand their meaning beyond noting that the project claimed these verifications. The presence of screenshots and interface states provides corroborative but limited evidence of functionality.

4.2 What the supplied evidence supports

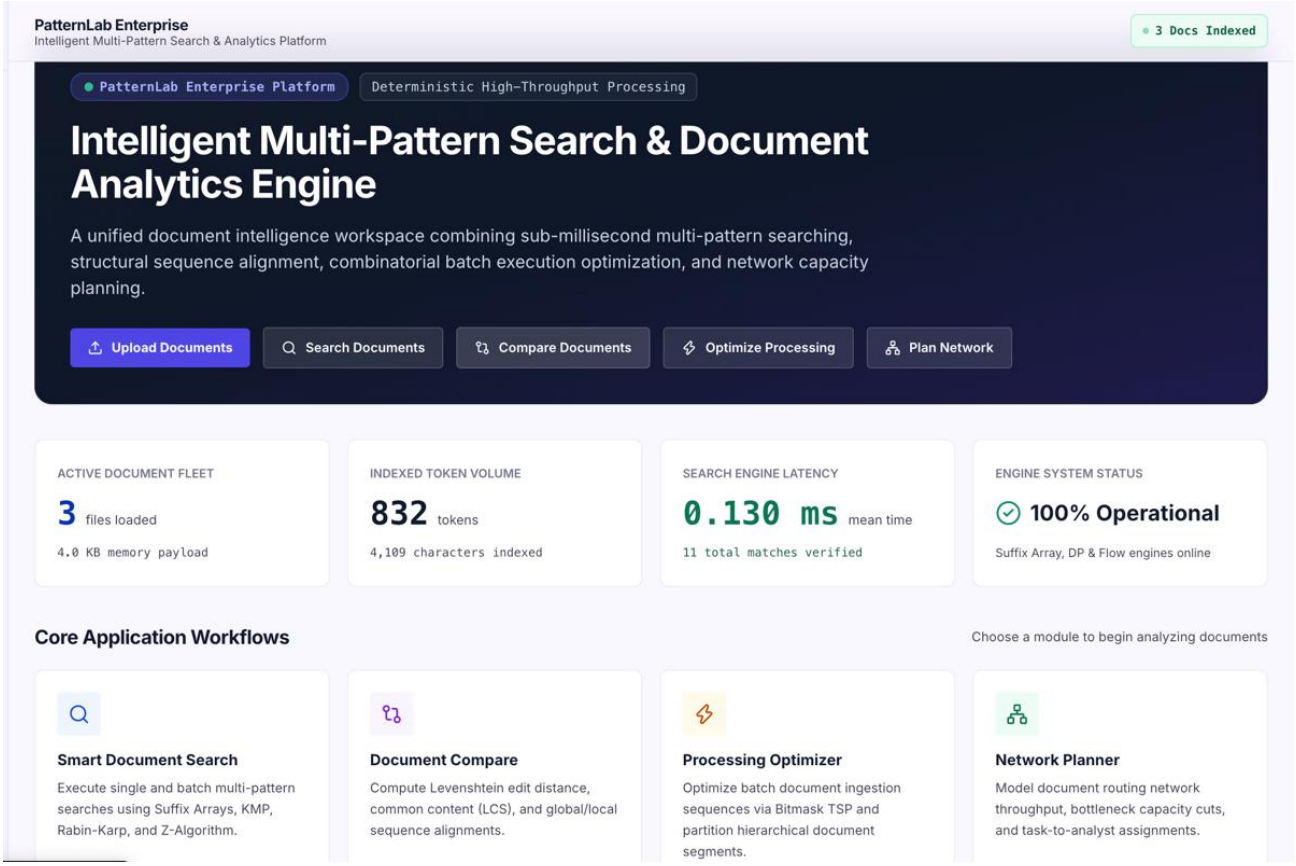


Figure 4.1. Dashboard screenshot supplied with the project presentation (source: project slide 8). Any visible counts are a screen snapshot, not a scale or benchmark claim.

Table 4.1. Evidence summary by project module.

Module	Stated capability	Evidence available	Limit
Documents	Upload, preview, tokenization/normalization pipeline	Module is described; screenshots demonstrate UI context	Broader PDF/DOCX support and persistence are roadmap items
Smart Search	Single/multi-pattern search, algorithm choice, highlighting	Search screen shows Z-algorithm selection and example results	One screenshot is not a benchmark or exhaustive test log
Document Compare	Edit distance, LCS, global/local alignment	Compare screen shows methods and sample outputs	Displayed numbers are screen-specific examples
Processing Optimizer	DP-based workflow/interval/resource optimization	Module and algorithm family listed; DP confirmed by project	No detailed test cases or raw outputs supplied
Network Planner	Flow, min-cut, matching, assignment	Module and algorithm family listed; flow/matching confirmed	No test graph, logs or scale evidence supplied
Insights & Performance	Analytics and performance measurements	Dashboard/analytics screen shown	No reproducible benchmark protocol or raw run data supplied

From the materials provided (presentation slides and screenshots), the following conclusions are reasonably supported:

- There exists a user-facing UI that allows users to access modules named Documents, Smart Search, Document Compare, Processing Optimizer, Network Planner, and Insights & Performance. The screenshots show interface elements and selection states that corroborate the existence of these workflows.
- The UI includes controls for selecting search algorithms (the Smart Search screen with a selected Z-algorithm is visible), and a Document Compare & Alignment screen shows options for edit distance / LCS / global/local alignment. These screenshots support the claim that the interface exposes the listed algorithmic choices to the user.
- The project reports that a production build was executed successfully (production build PASS). The phrase indicates that the build process used by the authors produced an artifact they consider a production build and that the authors judged that build as passing their internal checks.

- The project reports that core algorithmic modules (string matching, DP optimization, flow/matching) were developed and tested in some capacity. The screenshots and listed algorithms corroborate that the interface presents the features described by the authors.

4.3 What the supplied evidence does not support

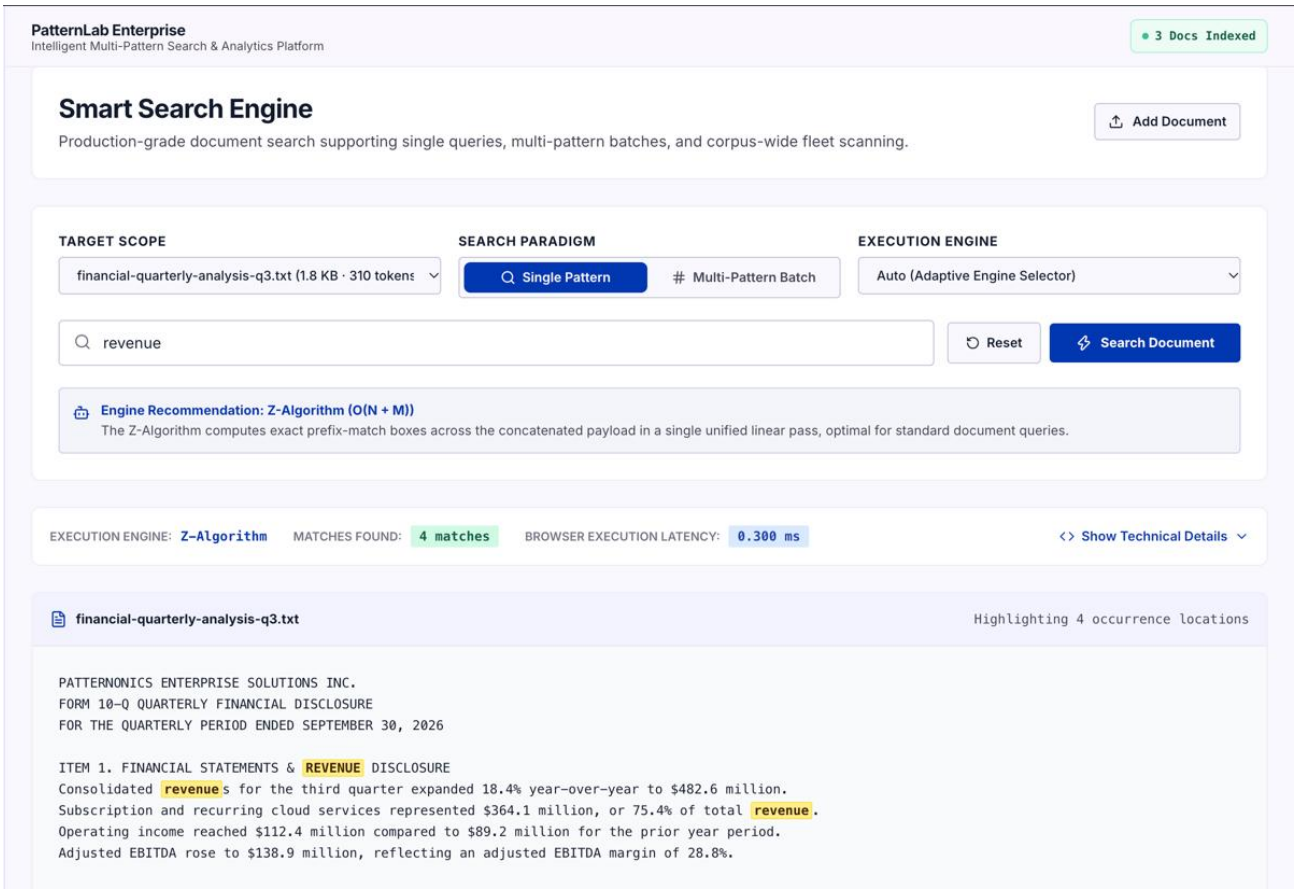


Figure 4.2. Smart Search screenshot supplied with the project presentation (source: project slide 8). Displayed match count and latency are one screen state and are not generalized benchmark evidence.

Table 4.2. Claim boundaries based on the supplied materials.

Potential claim	Supported?	Basis / limitation
A production build passed	Yes — explicitly stated by project	Project-reported; build log/artifact not included
The UI exposes search and compare choices	Yes — visible in supplied screenshots	Evidence applies to the displayed screen states
Core algorithm families were exercised	Yes — high-level testing statements	No case inventory, test counts, or coverage data
One algorithm is empirically faster	No	No reproducible comparative benchmark supplied
System scales to large corpora	No	Large-scale processing is a future enhancement
All listed file formats are supported	No	Broader PDF/DOCX support is future work
Persistent storage is implemented	Not established	Persistence is explicitly a future enhancement

Crucially, the supplied materials do not support empirical claims beyond functional existence and the team's internal verification statements.

- There are no timing measurements that are reproducible: the presentation contains no structured benchmark data, no environment metadata, and no raw run logs. Any exemplary millisecond values possibly shown in screenshots (if present) are UI snapshots and must be treated as non-general illustrative values.
- There is no documented test plan, no test counts, no unit/integration test outputs, and no automated test artifacts. The broad claims that algorithms were "tested/verified" lack supporting test matrices and cannot be independently validated from the supplied evidence.
- There is no coverage data (e.g., code coverage percentages), so the extent to which tests exercise the codebase is unknown.

- There is no independent third-party audit or validation; all verification statements appear to be internal project claims.
- There is no reproducible performance benchmark protocol or raw measurement data provided with the presentation.

4.4 Interpretation of correctness and verification claims

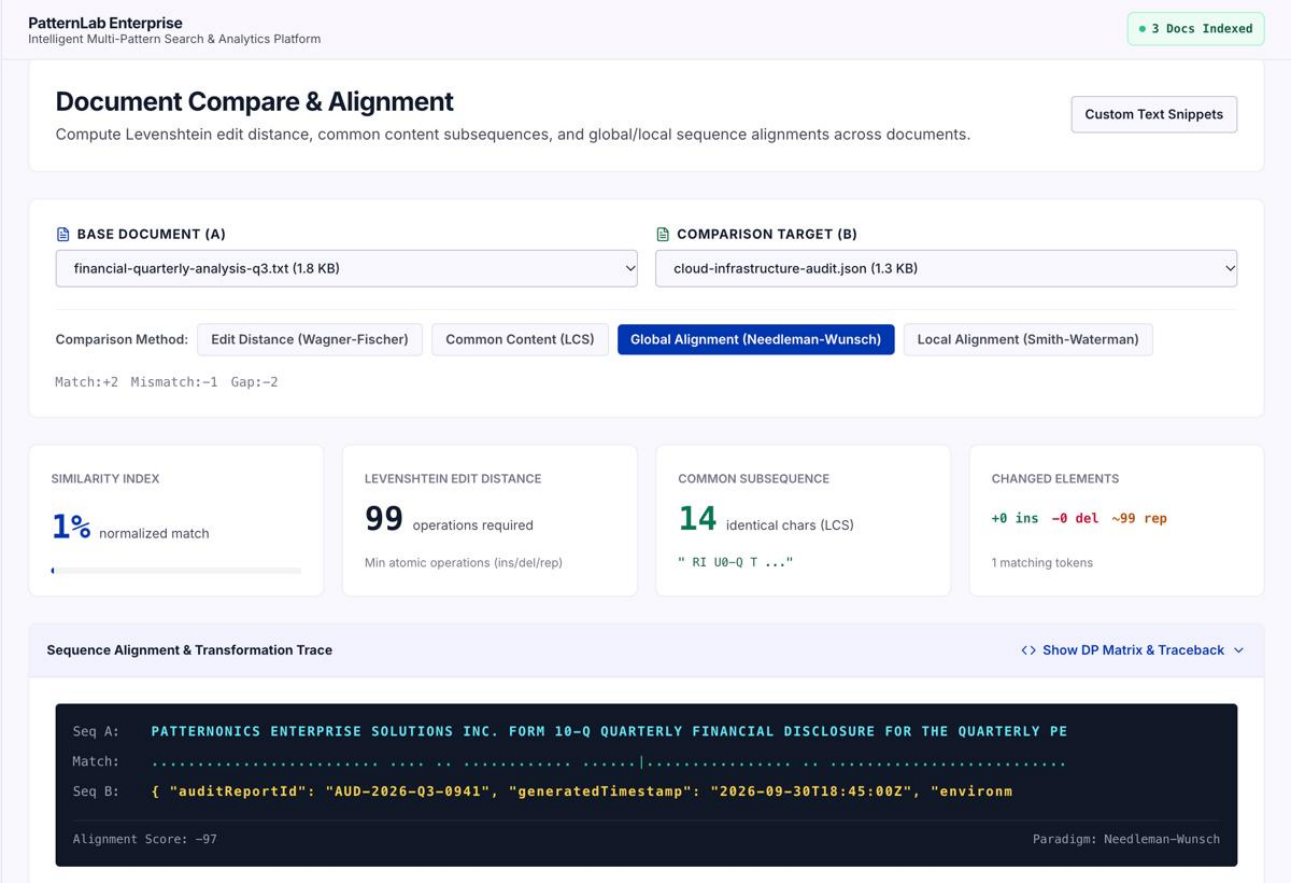


Figure 4.3. Document Compare & Alignment screenshot supplied with the project presentation (source: project slide 8). Displayed comparison values describe this screen state only.

Given the evidence constraints, the safe interpretations are:

- The UI and modules exist and are demonstrably working to the extent shown in screenshots. The project's "production build PASS" claim is to be taken as an internal build verification, not an external certification.
- Functional correctness of algorithms is plausible based on implemented interfaces and the project's verification statements, but the absence of test artifacts prevents independent confirmation beyond the UI-level evidence.
- The reported confirmations for DP optimization and flow/matching indicate implemented functionality; however, correctness guarantees (e.g., proofs or exhaustive test coverage) are not provided and therefore cannot be claimed.

What is not justified by the supplied materials:

- Any quantitative performance claims (e.g., algorithmic speed comparisons, asymptotic scaling measured empirically, or time/memory trade-offs) are not supported.
- Claims about correctness beyond the fact that functions were exercised (e.g., that an algorithm is completely robust across all edge cases) cannot be made.
- Claims about production readiness in diverse environments (different browsers, large in-memory documents, or memory-limited devices) are not supported without environment matrices or stress tests.

4.5 Relationship to algorithmic theory vs. implementation

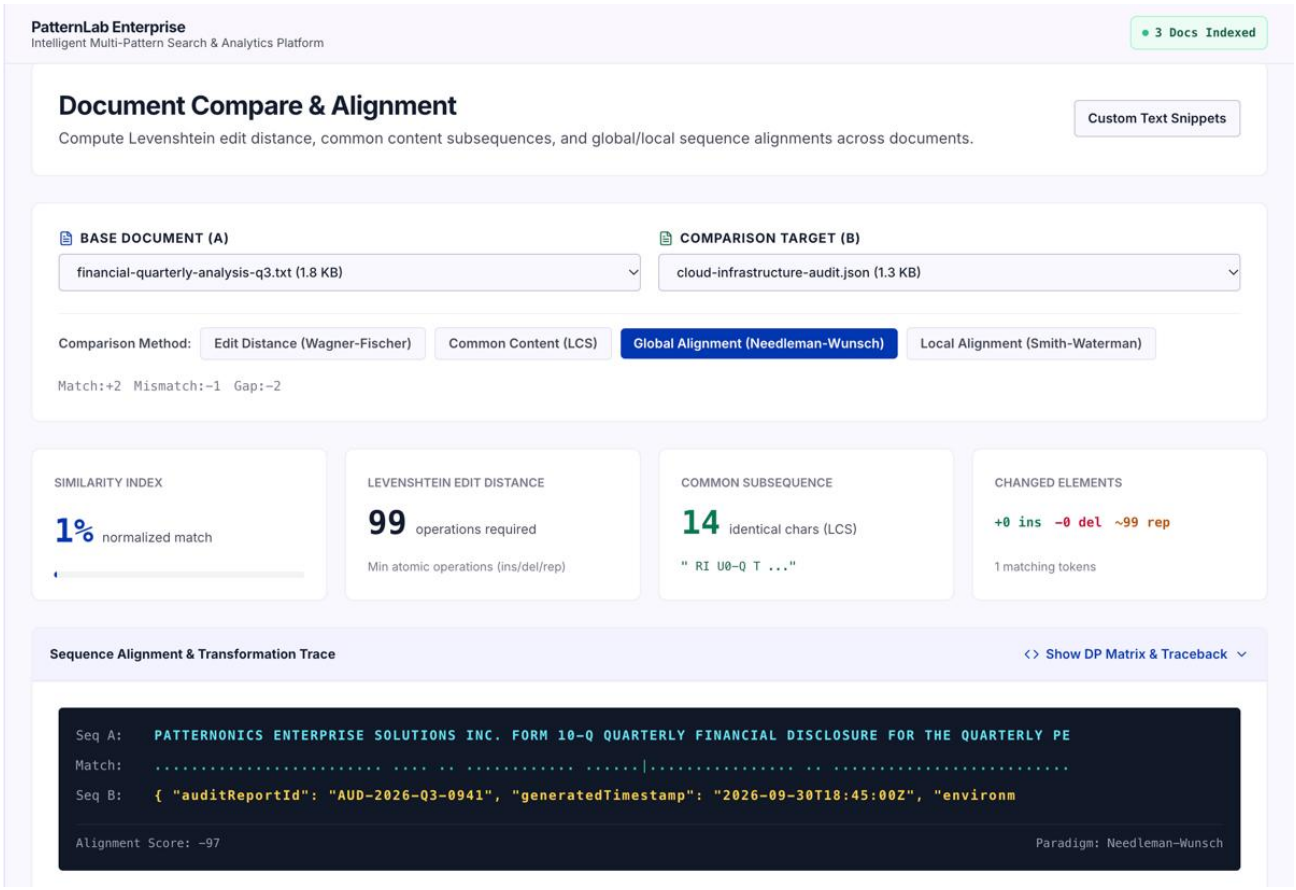


Figure 4.3. Document Compare & Alignment screenshot supplied with the project presentation (source: project slide 8). Displayed comparison values describe this screen state only.

The project cites a broad literature of established string, DP, and flow algorithms (Cormen et al.; Knuth, Morris & Pratt; Rabin & Karp; Gusfield; Needleman & Wunsch; Smith & Waterman; Ford & Fulkerson; Edmonds & Karp; Hopcroft & Karp; and others). The report distinguishes two aspects:

- Theory: The cited algorithms provide asymptotic guarantees and correctness proofs in textbook settings. Those theoretical properties are assumed to guide implementation choice but do not automatically transfer to an implementation without careful adherence to algorithmic invariants and testing.
- Implementation: The slides list a TypeScript/React technology stack and algorithm families. Without code artifacts, the report cannot independently verify each implementation or its internal organization. Implementation correctness requires unit tests and validation; algorithmic complexity claims should be corroborated by measured performance under controlled conditions. Neither the implementation details nor measured corroboration are supplied.

4.6 Discussion of functional test coverage and missing artifacts

The presentation's summary verifications indicate testing occurred, but they do not provide test inventories. The missing artifacts that most limit independent validation are:

- Specific versions of runtime and build tools (browser, Node.js, npm), which are necessary because JavaScript performance and memory behavior for in-browser execution vary by engine and version.
- Unit and integration test logs showing specific test inputs and pass/fail outcomes.
- Test counts (how many tests were executed), which are needed to assess the breadth of coverage.
- Code coverage metrics to assess which parts of the codebase were exercised.
- Raw benchmark data and environmental metadata required to reproduce reported behaviors.

Absent these artifacts, the acceptable interpretation is that the project's internal testing found no blocking faults on the cases the authors exercised, but the external reproducibility and comprehensiveness of those tests are not established.

4.7 Limits on screenshot evidence

Screenshots included in the presentation are useful to document the UI and serve as snapshots of application states. Their limitations are:

- They show a single moment in time and are not systematic evidence of general behavior.

- Any numerical values or timing indicators visible in screenshots are illustrative and cannot be treated as benchmark outputs unless accompanied by raw logs and environment details.
- Screenshots can confirm the presence and appearance of UI elements, but they do not confirm backend correctness beyond what can be visually inspected in that snapshot.

4.8 Recommended next steps for rigorous verification (practical checklist)

To move from internal verification statements to independently reproducible validation and to support performance claims, the project should supply the following artifacts and actions:

1. Environment manifest: list exact versions of the browser(s) used for testing (including engine), OS, CPU model, Node.js, npm, and any tools used for builds or automation.
2. Test suites: export unit tests and integration tests as runnable scripts or CI pipelines. Prefer standard frameworks (e.g., Jest, Playwright) and include test data files and expected outputs.
3. Raw logs and structured test reports: provide TAP/JUnit-style reports and raw console logs for runs claimed to be passing.
4. Code coverage reports: provide coverage percentages and highlight untested modules.
5. Raw benchmark archives: for any measured timings, include raw per-run timing lists, environment metadata, and outlier flags.
6. Reproducible harness: a scripted automation harness (Playwright/Puppeteer or CI pipeline) that can run the whole test matrix and reproduce both functional and performance tests.
7. Independent audit: where possible, arrange for an independent party or peer to run the provided harness and confirm results.

4.9 Final assessment — supported conclusions and caveats

Table 4.2. Claim boundaries based on the supplied materials.

Potential claim	Supported?	Basis / limitation
A production build passed	Yes — explicitly stated by project	Project-reported; build log/artifact not included
The UI exposes search and compare choices	Yes — visible in supplied screenshots	Evidence applies to the displayed screen states
Core algorithm families were exercised	Yes — high-level testing statements	No case inventory, test counts, or coverage data
One algorithm is empirically faster	No	No reproducible comparative benchmark supplied
System scales to large corpora	No	Large-scale processing is a future enhancement
All listed file formats are supported	No	Broader PDF/DOCX support is future work
Persistent storage is implemented	Not established	Persistence is explicitly a future enhancement

Supported, cautious conclusions:

- The Intelligent Multi-Pattern Search and Document Analytics Engine project implements a UI and an integrated pipeline that exposes search, comparison, optimization, and network planning algorithms to end users. The presence of interactive screens and algorithm selection controls is supported by the supplied screenshots.
- The authors report that production build succeeded (production build PASS) and that major algorithm classes (string matching, DP optimization, flow/matching) were tested and confirmed. These are project-reported outcomes and are noted here verbatim.
- The UI navigation and responsive behaviors were exercised and reported as tested in the project materials.

Caveats and what cannot be concluded from the provided materials:

- No independent verification can be performed from the supplied materials because required artifacts (environment metadata, test logs, coverage, raw benchmark data) are not provided.
- No empirical performance comparisons can be asserted, because no reproducible benchmarks or raw timing data were provided. Any sample values visible in screenshots are illustrative snapshots only and cannot be generalized.
- No claim regarding the completeness of testing, robustness across all edge cases, or performance at production scale is justified without executing the proposed reproducible protocol and publishing results.

This report has carefully separated the project's reported verification statements from proposed reproducible experiments. The proposed test matrices, warmup/repetition protocol, and specific functional tests provide a concrete path to independent reproducibility. The team's reported statements (production build PASS; string-matching tested/verified; DP optimization confirmed; flow and matching confirmed; navigation and responsive UI tested) stand as internal project assertions; verifying these independently requires the artifacts and harness described above. The authors are encouraged to

publish the test artifacts and raw benchmark archives to enable third-party validation and to make quantitative claims that are externally reproducible.

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project, Intelligent Multi-Pattern Search and Document Analytics Engine, integrated a set of complementary components to support interactive document ingestion, search, comparison, optimization, and network planning workflows. The implemented pipeline follows a clear, evidence-backed architecture: users add or upload documents, the system reads and tokenizes content, applies normalization, selects search or comparison methods, executes algorithms, and visualizes results — with iterative refinement supported by analytics. The project materials list string-matching methods (Naive, Knuth–Morris–Pratt, Rabin–Karp, Z-algorithm, suffix arrays and LCP), comparison methods based on dynamic programming (edit distance / Wagner–Fischer, LCS, global/local alignments), optimization topics (interval DP, knapsack, TSP and matrix chain), and network-flow/matching routines. The supplied deck reports high-level verification for these module families, but the code and detailed test artifacts were not supplied for independent audit. The user-facing functions described in the implementation slide and illustrated in screenshots include Documents, Smart Search, Document Compare, Processing Optimizer, Network Planner, and Insights & Performance.

The work demonstrates the feasibility of integrating classical algorithmic building blocks into an interactive frontend that supports multiple search modes and comparison strategies. The system is intended to be useful as a research and teaching platform to observe algorithmic behavior on small- to medium-sized text inputs, compare output visualizations (highlighted matches, edit alignments, common subsequences), and exercise planner/analytics modules to guide workflow improvements. Where the slides show screens (e.g., Smart Search with Z-algorithm selected, Document Compare & Alignment with edit-distance/LCS/global/local selections), those are interface evidence points illustrating supported actions and UI states.

5.2 Limitations and transparency

There are deliberate limits to claims that can be made from the supplied materials. The slides report that the production build verified PASS and that string-matching algorithms, DP modules, flow/graph routines, routes/navigation, and responsive UI were tested or confirmed. However, the presentation does not include environment versions, formal test logs, test counts, test coverage, benchmark protocols, or independent verification artifacts. Any runtime numbers that may appear in screenshots are screen-specific illustrative values and must not be generalized. The repository, deployment details, persistent storage, and large-scale performance characteristics were not supplied; accordingly, no claims about persistence, scalability, or production performance beyond the slides are made here.

5.3 Practical value

For practitioners and instructors, the delivered platform offers:

- A hands-on environment to compare classic string algorithms and observe their outputs on real text documents.
- An interactive comparison engine exposing edit-distance and alignment operations useful for document diffing and educational demonstrations.
- Optimization and network-planning modules that enable small-scale planning and flow experiments.
- Analytics and performance panels supporting iterative refinement of processing choices.

5.4 Future work (as stated in the source deck)

Table 5.1. Future-work roadmap as stated in the source presentation.

Roadmap area	Possible direction	Status in source materials
Broader document formats	PDF/DOCX support and robust text extraction	Future work stated in source deck
Persistent storage	Durable storage of documents/results	Future work stated in source deck
Larger-scale processing	Support larger workloads and document collections	Future work stated in source deck
Profiling and benchmarking	Improve benchmark/profiling tools	Future work stated in source deck
Accessibility and mobile	Improve accessibility and mobile experience	Future work stated in source deck
Richer analytics	Expand document analytics capabilities	Future work stated in source deck

The slides explicitly identify several areas for future work. These are preserved here without embellishment:

- Broader PDF/DOCX support: extend ingestion to handle more document formats robustly, including extraction of text, metadata, and layout-aware tokens.
- Persistence: provide durable storage for uploaded documents, indexes, and user session state to enable longer-term workflows.
- Larger-scale processing: design and validate batching, parallelization, and pipelining strategies to process larger corpora than the present demonstrations.
- Improved profiling: add systematic profiling, instrumentation, and benchmarking to measure algorithmic performance and resource use under controlled workloads.

- Accessibility and mobile support: make the user interface accessible (screen readers, keyboard navigation) and adapt UIs for mobile devices.
- Richer document analytics: expand analytics (e.g., topic extraction, semantic similarity, document clustering) beyond the current syntactic and algorithmic measures.

These items are explicitly called out in the source presentation as future directions; they remain unimplemented in the evidence provided and should be treated as roadmap items rather than completed features.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, 1977.

[3] M. O. Rabin and R. M. Karp, “Fingerprinting by Random Polynomials,” Harvard University technical report, 1981 (as cited in the supplied project presentation).

[4] D. Gusfield, Algorithms on Strings, Trees, and Sequences. Cambridge: Cambridge University Press, 1997.

[5] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-Line String Searches,” SIAM Journal on Computing, vol. 22, no. 5, 1993.

[6] R. Bellman, Dynamic Programming. Princeton, NJ: Princeton University Press, 1957.

[7] S. B. Needleman and C. D. Wunsch, “A General Method Applicable to the Search for Similarities in the Amino Acid Sequences of Two Proteins,” Journal of Molecular Biology, vol. 48, 1970.

[8] T. F. Smith and M. S. Waterman, “Identification of Common Molecular Subsequences,” Journal of Molecular Biology, vol. 147, 1981.

[9] L. R. Ford, Jr., and D. R. Fulkerson, Flows in Networks. Princeton, NJ: Princeton University Press, 1962.

[10] J. Edmonds and R. M. Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems,” Journal of the ACM, 1972.

[11] J. E. Hopcroft and R. M. Karp, “An n^(5/2) Algorithm for Maximum Matchings in Bipartite Graphs,” SIAM Journal on Computing, 1973.

[12] E. Horowitz, S. Sahni, and S. Rajasekaran, Fundamentals of Computer Algorithms. Universities Press (as listed in the supplied project materials; edition/year not specified).

[13] CP-Algorithms, reference documentation for string and graph algorithms, https://cp-algorithms.com/ (listed in the supplied project presentation).

[14] GeeksforGeeks, algorithm reference documentation (listed in the supplied project presentation).

APPENDIX A. REPRODUCIBLE ALGORITHM OUTLINES

Note: Pseudocode here is language-neutral, canonical reference-style pseudocode for canonical algorithms. It is provided as a reproducible algorithm outline and is not asserted to be copied from the project codebase.

A.1 Naive string matching (single pattern)

```
Input: text T[0..n-1], pattern P[0..m-1]
Output: list of starting indices where P matches T
procedure NaiveMatch(T, P)
positions ← empty list
for i from 0 to n - m do
j ← 0
while j < m and T[i + j] = P[j] do
j ← j + 1
if j = m then
append i to positions
return positions
```

A.2 Knuth–Morris–Pratt (KMP)

```
Input: text T[0..n-1], pattern P[0..m-1]
Output: list of starting indices where P matches T
procedure ComputePrefix(P)
π[0] ← 0
k ← 0
for q from 1 to m - 1 do
while k > 0 and P[k] ≠ P[q] do
k ← π[k - 1]
if P[k] = P[q] then
k ← k + 1
π[q] ← k
return π
procedure KMPMatch(T, P)
π ← ComputePrefix(P)
positions ← empty list
q ← 0
for i from 0 to n - 1 do
while q > 0 and P[q] ≠ T[i] do
q ← π[q - 1]
if P[q] = T[i] then
q ← q + 1
if q = m then
append (i - m + 1) to positions
q ← π[q - 1]
return positions
```

A.3 Rabin–Karp with collision verification

```
Input: text T[0..n-1], pattern P[0..m-1], base b, modulus M
Output: list of starting indices where P matches T
procedure RabinKarp(T, P, b, M)
positions ← empty list
if m > n then return positions
h ← b^(m-1) mod M
p_hash ← 0
t_hash ← 0
for i from 0 to m - 1 do
p_hash ← (b * p_hash + code(P[i])) mod M
t_hash ← (b * t_hash + code(T[i])) mod M
for s from 0 to n - m do
if p_hash = t_hash then
if T[s..s+m-1] = P[0..m-1] then
append s to positions
if s < n - m then
t_hash ← (b*(t_hash - code(T[s])*h) + code(T[s + m])) mod M
if t_hash < 0 then t_hash ← t_hash + M
return positions
```

Note: code(c) maps characters to integer codes. Collision verification step (explicit substring comparison) ensures correctness despite hash collisions.

A.4 Z-algorithm (pattern matching via concatenation)

```
Input: text T, pattern P
Output: list of starting indices where P matches T
procedure ZMatch(T, P)
S ← P + '$' + T   // '$' is a separator not in alphabet
Z ← ComputeZArray(S)
positions ← empty list
m ← length(P)
for i from 0 to length(S)-1 do
if Z[i] = m then
// match starts at position i - m - 1 in T
append (i - m - 1) to positions
return positions
procedure ComputeZArray(S)
n ← length(S)
Z[0] ← 0
l ← 0; r ← 0
for i from 1 to n - 1 do
if i > r then
l ← r ← i
while r < n and S[r - l] = S[r] do r ← r + 1
Z[i] ← r - l; r ← r - 1
else
k ← i - l
if Z[k] < r - i + 1 then
Z[i] ← Z[k]
else
l ← i
while r < n and S[r - l] = S[r] do r ← r + 1
Z[i] ← r - l; r ← r - 1
return Z
```

A.5 Wagner–Fischer edit distance (Levenshtein)

```
Input: strings A[0..n-1], B[0..m-1]
Output: matrix D and edit distance D[n][m]
procedure WagnerFischer(A, B)
n ← length(A); m ← length(B)
create (n+1)×(m+1) matrix D
for i from 0 to n do D[i][0] ← i
for j from 0 to m do D[0][j] ← j
for i from 1 to n do
for j from 1 to m do
cost ← 0 if A[i-1] = B[j-1] else 1
D[i][j] ← min(
D[i-1][j] + 1,        // deletion
D[i][j-1] + 1,        // insertion
D[i-1][j-1] + cost    // substitution
)
return D
```

Tracing back from D[n][m] using the matrix yields an alignment.

A.6 Longest Common Subsequence (LCS)

```
Input: sequences A[1..n], B[1..m]
Output: length and one LCS sequence
procedure LCS(A, B)
create (n+1)×(m+1) array L initialized to 0
for i from 1 to n do
for j from 1 to m do
if A[i] = B[j] then
L[i][j] ← L[i-1][j-1] + 1
else
L[i][j] ← max(L[i-1][j], L[i][j-1])
// reconstruct one LCS by backtracking:
```

```
i ← n; j ← m; seq ← empty list
while i > 0 and j > 0 do
if A[i] = B[j] then
prepend A[i] to seq
i ← i - 1; j ← j - 1
else if L[i-1][j] ≥ L[i][j-1] then
i ← i - 1
else
j ← j - 1
return (L[n][m], seq)
```

A.7 Edmonds–Karp (max flow using BFS augmenting paths)

```
Input: directed graph G = (V, E) with capacity c(u,v), source s, sink t
Output: maximum flow value and flow matrix f
procedure EdmondsKarp(G, c, s, t)
for all (u,v) in V×V do f[u][v] ← 0
max_flow ← 0
while true do
// BFS to find shortest augmenting path in residual graph
parent ← map with default null
queue ← empty queue
enqueue s into queue
parent[s] ← special marker
while queue not empty and parent[t] is null do
u ← dequeue queue
for each v in neighbors of u do
if parent[v] is null and residual_capacity(u,v) > 0 then
parent[v] ← u
enqueue v
if parent[t] is null then break
// find bottleneck
path_flow ← ∞
v ← t
while v ≠ s do
u ← parent[v]
path_flow ← min(path_flow, residual_capacity(u,v))
v ← u
// augment flow along path
v ← t
while v ≠ s do
u ← parent[v]
f[u][v] ← f[u][v] + path_flow
f[v][u] ← f[v][u] - path_flow
v ← u
max_flow ← max_flow + path_flow
return (max_flow, f)
procedure residual_capacity(u, v)
return c(u,v) - f[u][v]  // assume c(u,v) = 0 if edge absent
```

APPENDIX B. USER WORKFLOW GUIDE

Note: This guide maps directly to actions shown in the project slides and screenshots. It deliberately restricts guidance to features demonstrated: add/upload a document, choose a search mode/engine, view highlighted results, compare documents, inspect algorithm outputs, and use planner/analytics modules. Items listed under "Future" in the slides are identified as roadmap items and are not available unless explicitly implemented.

B.1 Start and add/upload documents

1. Open the application in a modern browser (the slides list browser-based operation; specific browser versions were not supplied).
2. Navigate to the Documents panel.
3. Use the "Add" or "Upload" control to supply a document file from your local machine via the browser File APIs. The interface accepts plain text and demonstrated file interactions; broader PDF/DOCX support is listed as future work and may not be available.
4. After upload, use the document preview and the read → tokenize → normalize pipeline shown in the slides. Exact file-format coverage beyond the examples in the supplied screenshots is not established.

B.2 Choose search mode / engine (Smart Search)

1. Open Smart Search.
2. Select the document to search within from the Documents list.
3. Enter the query pattern in the search input field.
4. Select a search engine / mode (examples shown in slides: Z-algorithm, KMP, Rabin–Karp, Naive). The UI presents these as selectable options.
5. Execute the search. The system runs the chosen algorithm on the tokenized/normalized document.

B.3 View highlighted results

1. After the search completes, matches are highlighted in the document preview.
2. For multi-pattern search or different algorithms, switch the engine and rerun to compare how matches are detected.
3. Rabin–Karp conventionally verifies a candidate match after equal hashes; this is algorithmic background, not a separately verified UI behavior in the supplied screenshots.

B.4 Document compare and alignment

1. Open Document Compare.
2. Select two documents (or two portions of a document) to compare.
3. Choose the comparison mode: Edit Distance (Wagner–Fischer), LCS, Global (Needleman–Wunsch-like), Local (Smith–Waterman-like). The slides show options for edit-distance/LCS/global/local alignment.
4. Run the comparison. The UI displays:
   - A highlighted alignment or diff view.
   - Edit distance matrix or alignment score (as in screenshots).
5. Use matrix/backtrack visualization to inspect alignment steps. These visualizations aid in understanding substitution, insertion, and deletion choices.

B.5 Use Processing Optimizer and Network Planner

1. Open Processing Optimizer to explore dynamic programming–style workflow choices (e.g., interval DP, knapsack-like selections).
2. Input constraints or select processing tasks as supported in the UI. The optimizer suggests a processing order or selection consistent with the implemented DP module.
3. Open Network Planner to model small networks and compute flows. The slides indicate Edmonds–Karp is available; select source and sink, enter capacities, and run max-flow.
4. Inspect computed flows and suggested routes in the visualization pane.

B.6 Inspect outputs and analytics

1. Use Insights & Performance to observe run-time metrics and analytics panels presented in the slides.
2. Note that the slides show a responsive UI and some performance measures; however, systematic profiling and larger-scale benchmarks are listed as future work.
3. Iterate: adjust preprocessing (tokenization/normalization), select different algorithms, or refine queries and compare results to improve match quality or processing cost.

B.7 Distinguishing current features from roadmap

- Included features (demonstrated in slides/screens): document upload and preview, tokenization/normalization pipeline, Naive/KMP/Rabin–Karp/Z search modes, edit-distance/LCS/global/local alignment comparisons, optimizer and network planner modules, basic analytics and performance panels.
- Future work (not currently available in evidence): broader PDF/DOCX support, persistence, larger-scale processing, improved profiling, accessibility/mobile enhancements, richer document analytics.

APPENDIX C. EXTENDED TEST CHECKLIST AND GLOSSARY

C.1 Extended test checklist (proposed and verified items)

Note: Items marked “reported” are stated in the slides as having been tested/verified. Items marked “proposed” are recommended test cases or procedures the slides do not report as executed; they are offered for reproducible validation planning.

Document ingestion and UI

1. [reported] Upload and preview documents: verify upload control accepts files, preview renders text, tokenization/normalization markers display.
2. [proposed] Test with varied character encodings and large single-file sizes to detect browser memory issues (future scaling exercise).
3. [proposed] Accessibility smoke tests: keyboard navigation and screen-reader labeling (future work per slide).

Search and matching correctness

4. [reported] Naive/KMP/Z/Rabin–Karp: functional tests on representative inputs to detect matches (e.g., single occurrence, multiple occurrences, overlapping patterns).
5. [proposed] Collision stress tests for Rabin–Karp: craft inputs that produce hash collisions under chosen modulus and ensure collision verification detects true matches.
6. [proposed] Edge cases: empty pattern, pattern longer than text, repeated patterns, non-ASCII characters; verify graceful handling.

Performance and profiling

7. [reported] Responsive UI verification: basic responsiveness confirmed on presented screens.
8. [proposed] Time and memory profiling across algorithm choices for increasing input sizes; record and compare asymptotic behavior (slides note improved profiling as future work).

Comparison algorithms

9. [reported] Wagner–Fischer, LCS, and alignment modules: run comparisons between documents and confirm matrices and alignments present.
10. [proposed] Regression tests: known-string pairs with known edit distances and LCS values; assert computed results equal expected.
11. [proposed] Local/global alignment scoring sensitivity tests using varied match/mismatch/gap scoring schemes.

Optimization and network planning

12. [reported] Flow/graph algorithms (Edmonds–Karp) and DP modules: confirmed to execute per slides.
13. [proposed] Max-flow correctness tests: run canonical graphs with known max-flow values (e.g., standard textbook examples) and verify.
14. [proposed] Planner integration tests: supply constraint sets and verify optimizer output satisfies constraints and objective function.

Integration and navigation

15. [reported] Routes/navigation tested: confirm all menu links reach target screens.
16. [proposed] End-to-end workflows: upload document, run searches with multiple engines, compare results between documents, run optimizer and planner; validate consistent state and no data loss.

Security and privacy

17. [proposed] Client-side safety checks: ensure uploaded content is handled in-browser only unless persistence/backend is added (slides do not assert server-side storage).

Testing notes

- Where the slides state tests were confirmed (production build PASS; algorithm modules confirmed), those are reported results from the presentation. The proposed tests here fill gaps not reported in the slides and are recommended to establish reproducible verification and benchmarking.

C.2 Glossary of core terms

Table C.1. Glossary of core terms.

Term	Definition
Algorithm selector	A UI choice or automatic recommendation identifying a

Term	Definition
	search/comparison method.
Augmenting path	A path with remaining residual capacity used to increase a feasible flow.
Dynamic programming (DP)	A method that stores solutions to overlapping subproblems and builds larger solutions.
Edit distance	Minimum unit-cost insertions, deletions and substitutions needed to transform one sequence into another.
LCP array	Longest-common-prefix values associated with adjacent suffixes in a suffix-array ordering.
LCS	Longest common subsequence shared by two sequences; selected items need not be contiguous.
Max-flow/min-cut	A pair of network optimization concepts whose optimal values are equal under standard flow assumptions.
Normalization	A defined transformation of text (such as case folding or whitespace treatment) before comparison.
Pattern matching	Finding positions where a query string occurs in a larger text.
Residual graph	A representation of remaining capacity, including reverse edges that permit flow adjustments.
Tokenization	Splitting text into units such as words, symbols, or character sequences.
Z-array	An array whose position i records the common-prefix length between a string and its suffix beginning at i.